

CI/CD Best Practices on Databricks

Databricks Notebooks

The Databricks Notebook

Multi-language

Use Python, SQL, Scala, and R, all in one Notebook

Collaborative

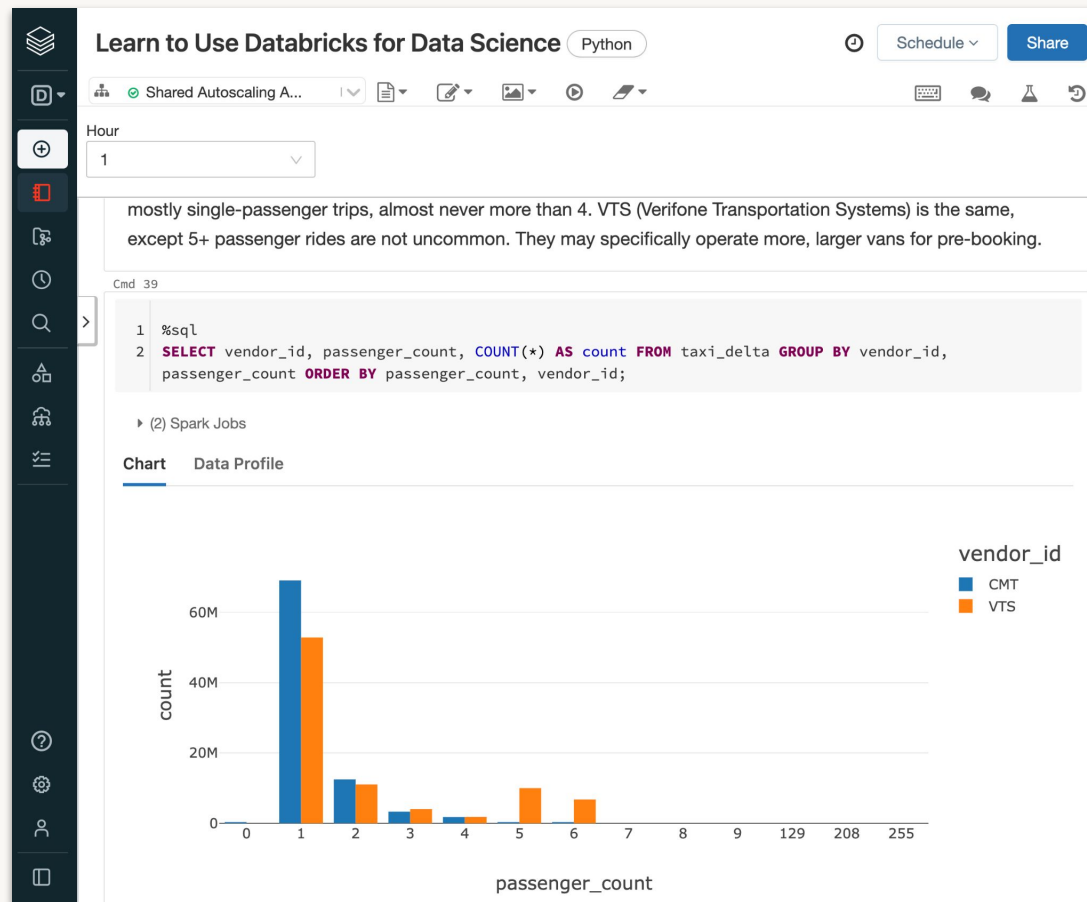
Real-time co-presence, co-editing, and commenting

Jupyter-compatible

Use the power of the Jupyter ecosystem in the Notebook

Ideal for exploration

Explore, visualize, and summarize data with built-in charts and data profiles



Adaptable

Install standard libraries and use local modules

Reproducible

Automatically track version history, and use git version control with Repos

Get to production faster

Quickly schedule notebooks as jobs or create dashboards from their results, all in the Notebook

Enterprise-ready

Enterprise-grade access controls, identity management, and auditability



Adaptable environments

Install libraries scoped to the Notebook

```
1 %pip install folium seaborn==0.11.1
```

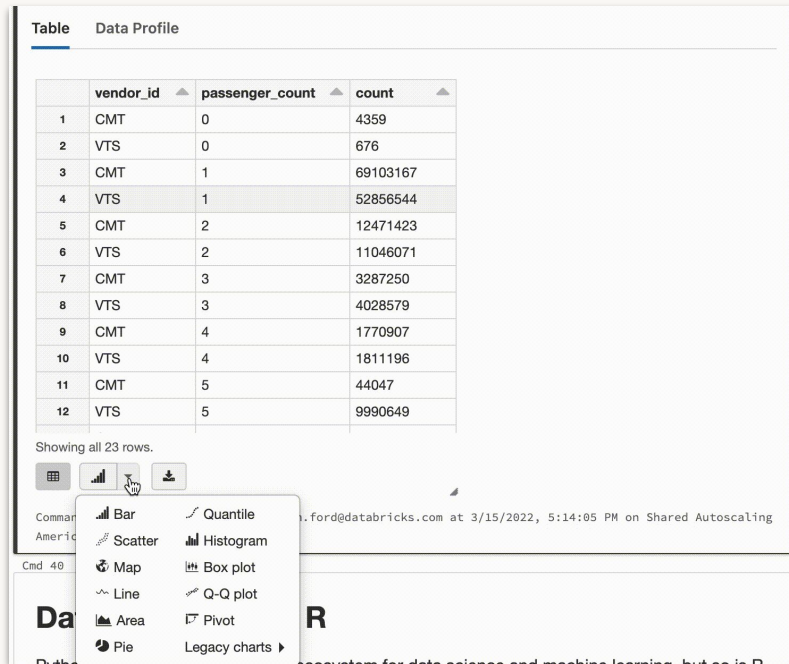
```
1 import seaborn as sns
2
3 sns.violinplot(data=tips, x="day", y="total_bill", hue="smoker",
4               split=True, inner="quart", linewidth=1,
5               palette={"Yes": "b", "No": ".85"})
6 sns.despine(left=True)
```

Install Python libraries for a notebook
without affecting other users with
%pip

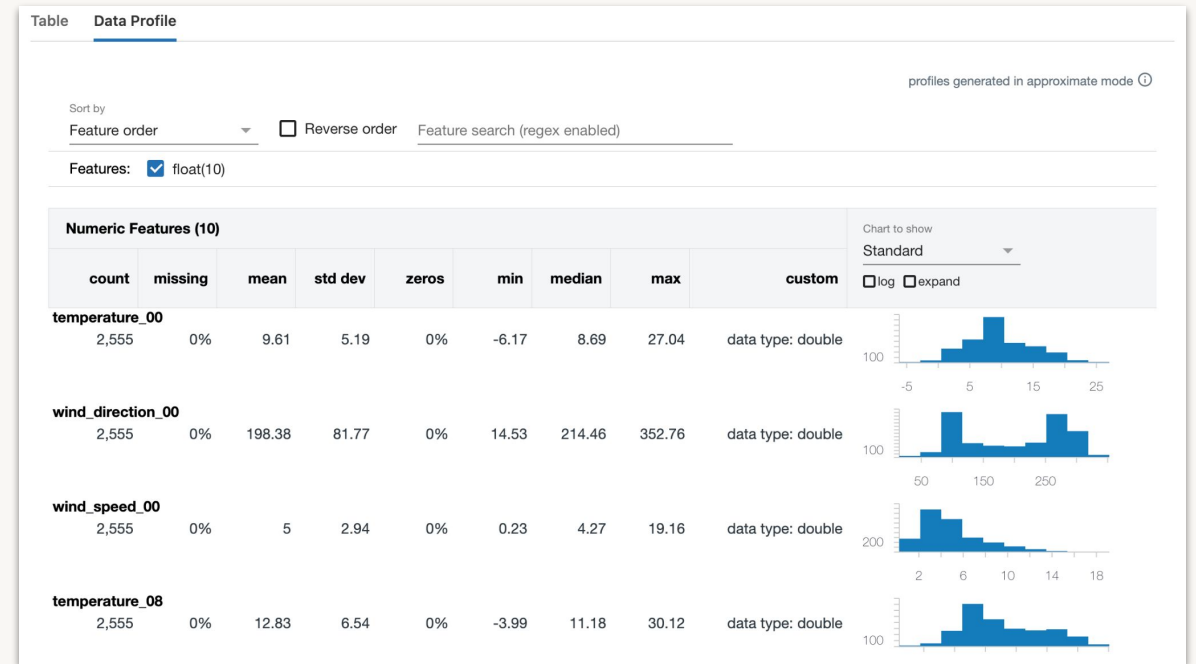


Explore data efficiently

Native tools for visualizing and understanding data



Create **interactive charts** to visualize data in the Notebook with only two clicks



Summarize a data set's essential properties and statistics in a **data profile** with the push of a button



SQL Warehouse in Notebooks

Use SQL Warehouses as a compute resource within the Notebook

Run SQL workloads against a SQL warehouse within the Notebook

Schedule Notebooks to run as Jobs using SQL warehouses

Create Notebook Dashboards that run queries using SQL warehouses

Attach to an existing compute resource

☐ General cluster ☒ SQL Warehouse

Data Team Endpoint

Summary

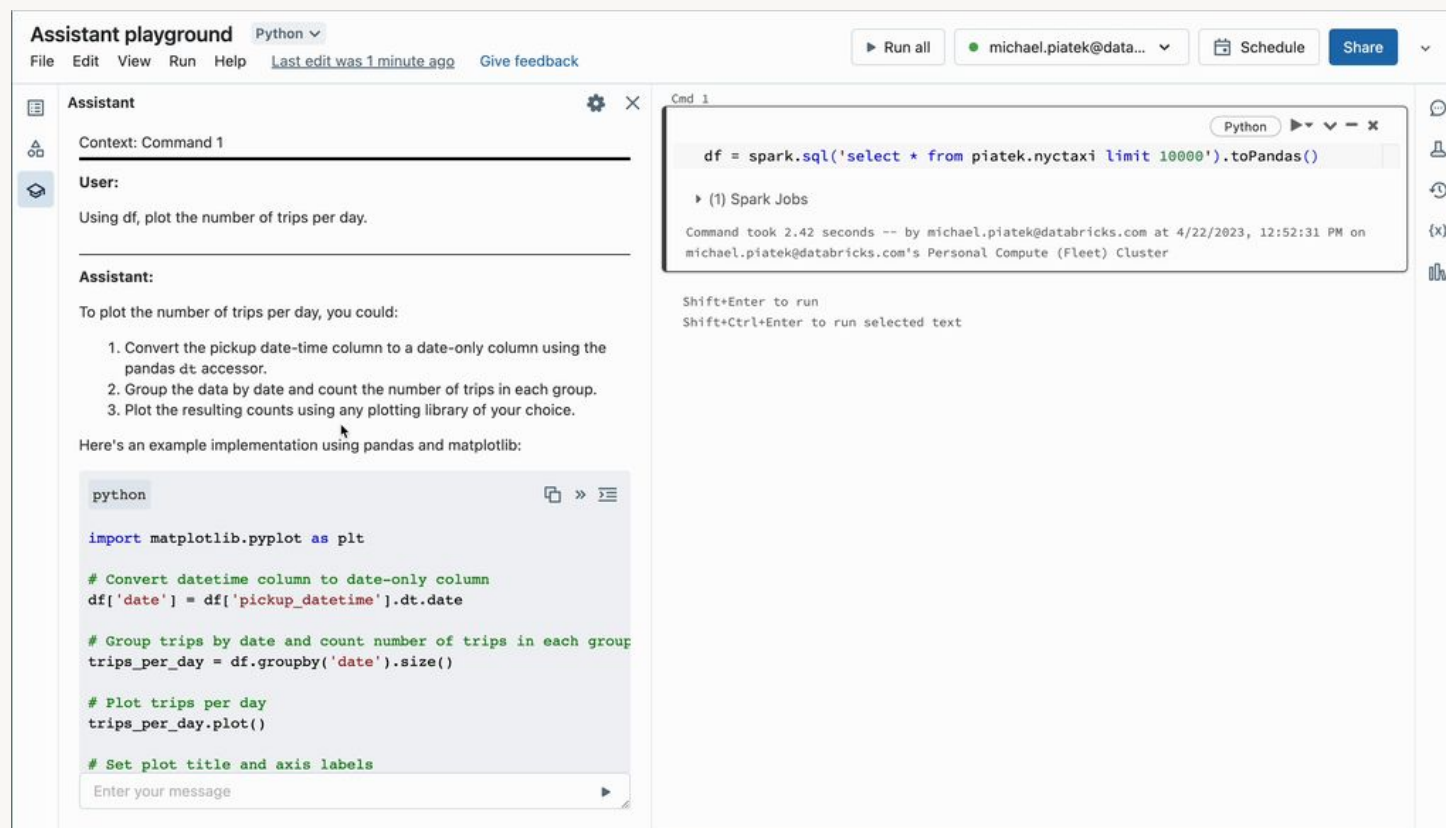
Size	Medium
Type	Serverless

Notebooks can only be attached to Pro or Serverless SQL Warehouses

CancelAttach

Context-Aware Authoring Assistant

- **Complete:** suggestions-as-you-type
- **Generate:** NL Prompt → code/SQL
- **Explain:** Highlight a query and get an explanation in plain English
- **Debug:** Explain and fix syntax and runtime errors in your code with a single click.
- **Transform/Optimize:** Convert languages; RDD → DataFrame; Pandas → PySpark; Performance improvements



IDE Support

Databricks Extension for Visual Studio Code

Simple Setup

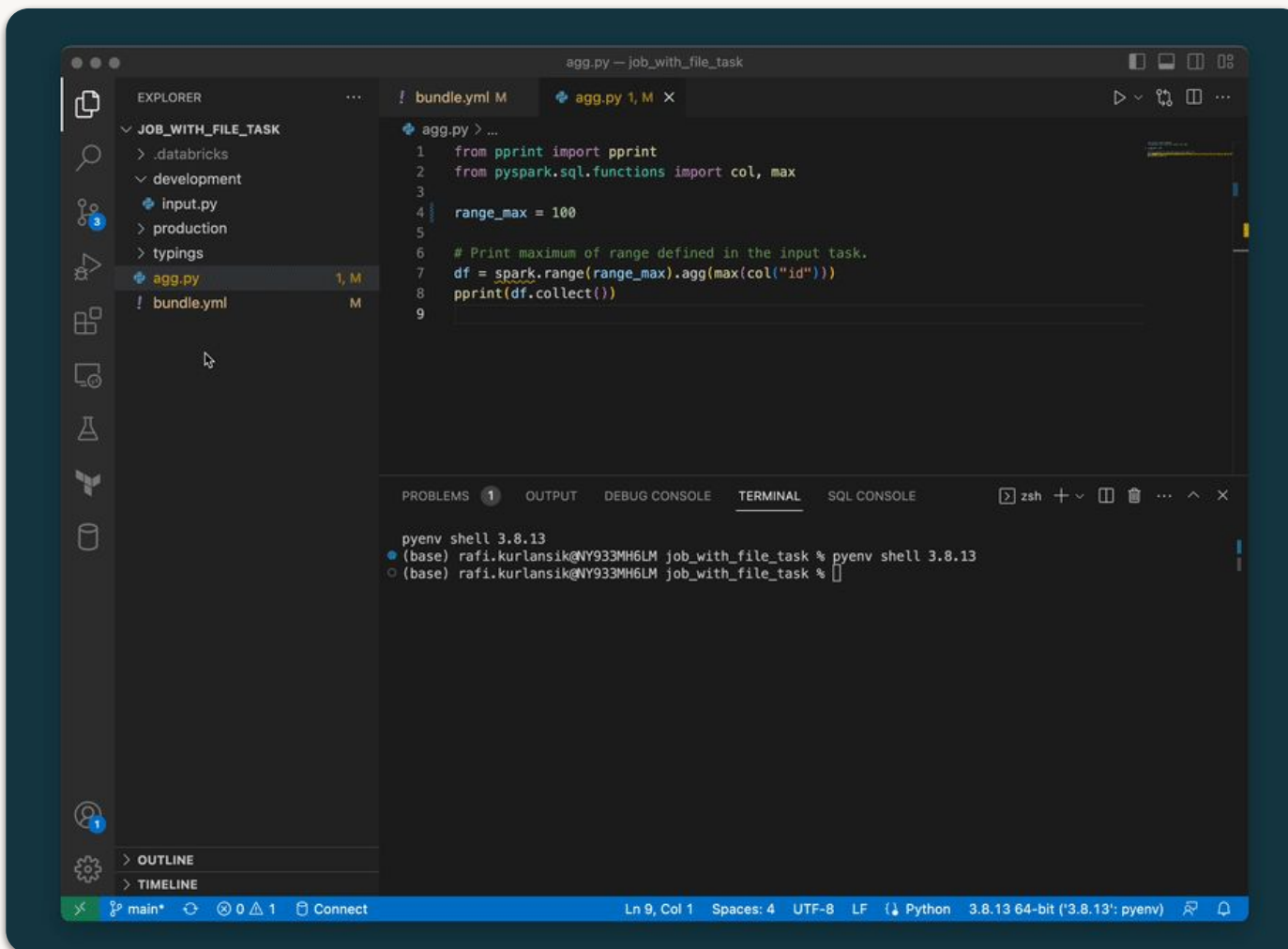
Find us on the VS Code Marketplace and get connected to compute in minutes

Native Experience

Write code using the productivity features you love from VS Code

Run on Databricks

Execute batch workloads or start interactively debugging directly from your IDE



Databricks Extension for Visual Studio Code

Simple Setup

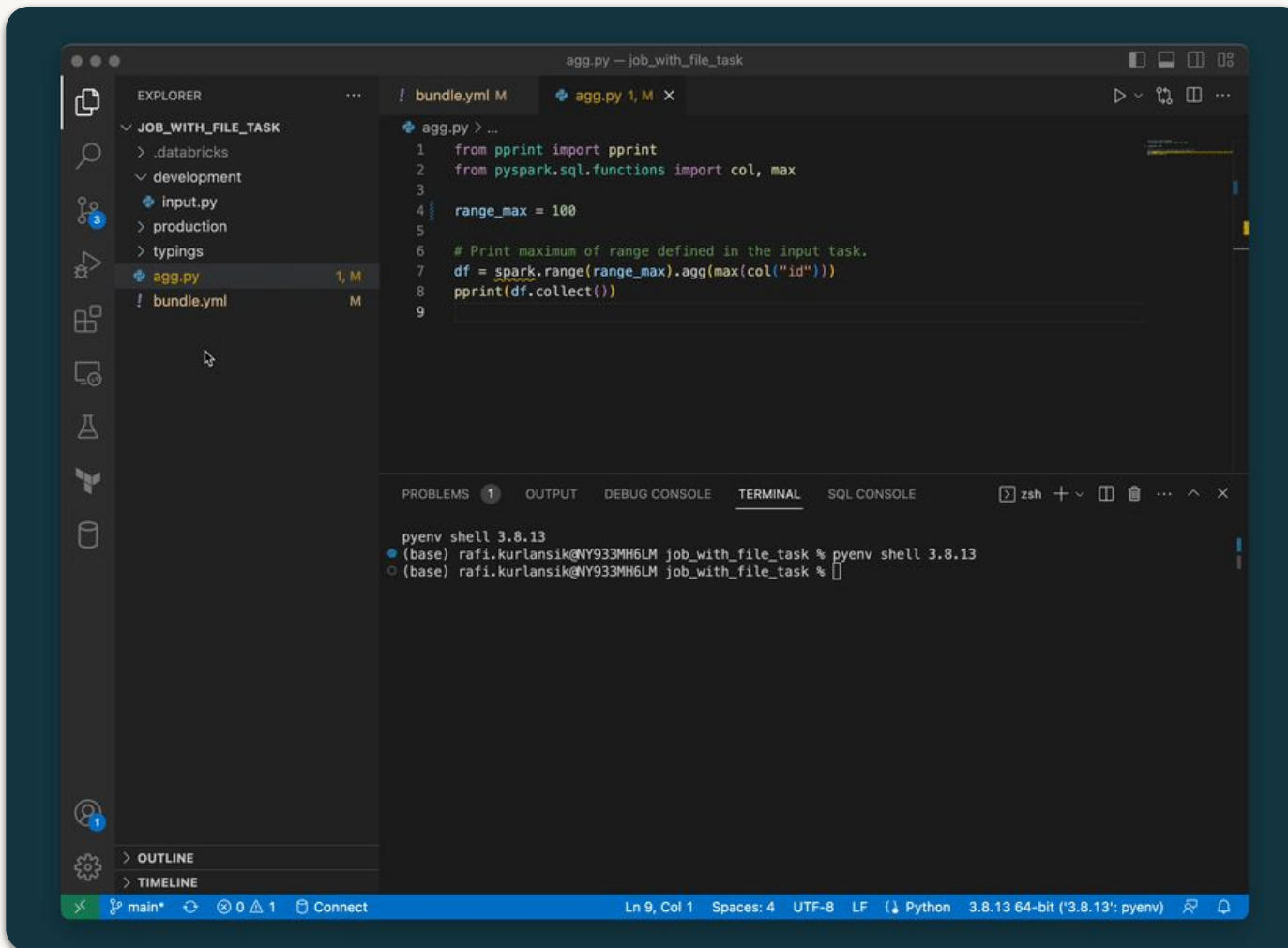
Find us on the VS Code Marketplace and get connected to compute in minutes

Native Experience

Write code using the productivity features you love from VS Code

Run on Databricks

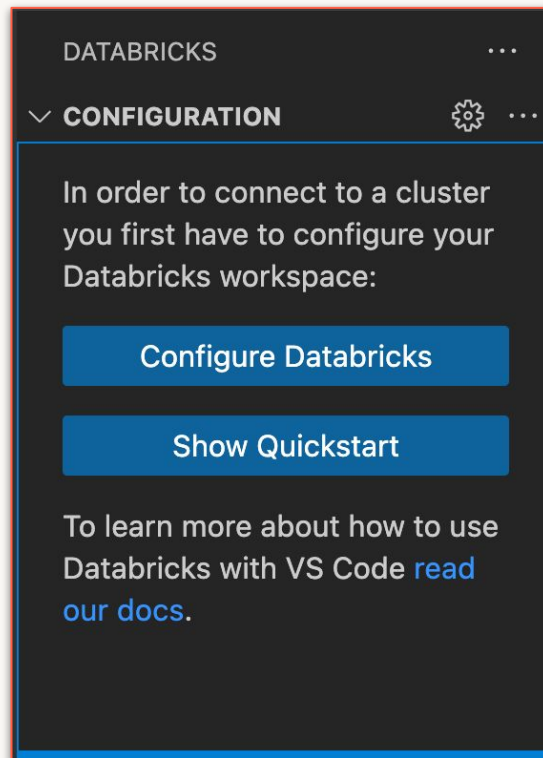
Execute batch workloads or start interactively debugging directly from your IDE



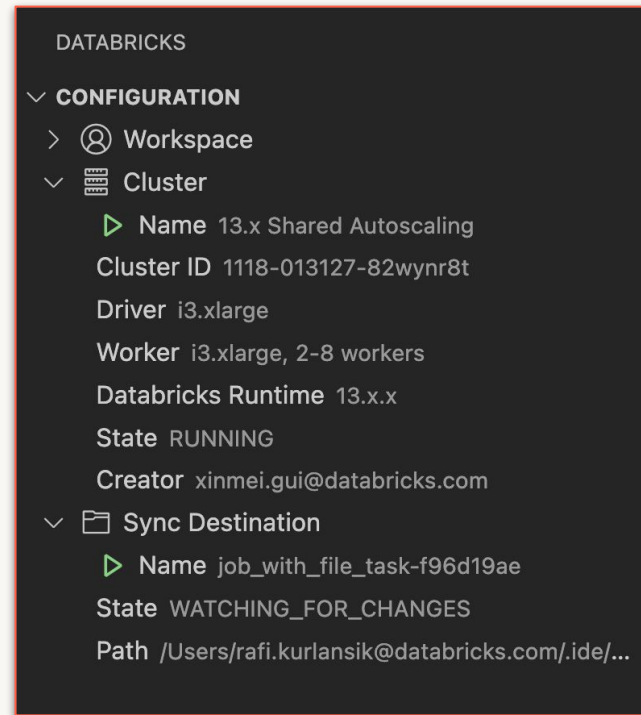
Databricks Extension for Visual Studio Code

Easy to configure, connect, and run code in a Workspace

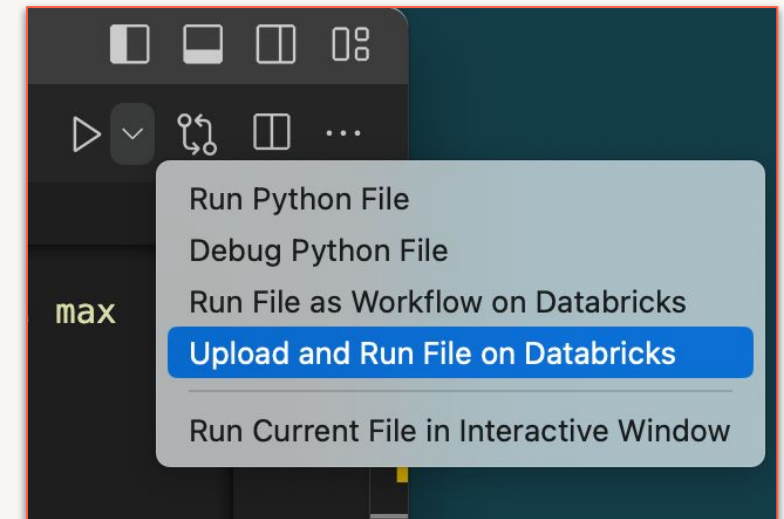
Configure



Connect



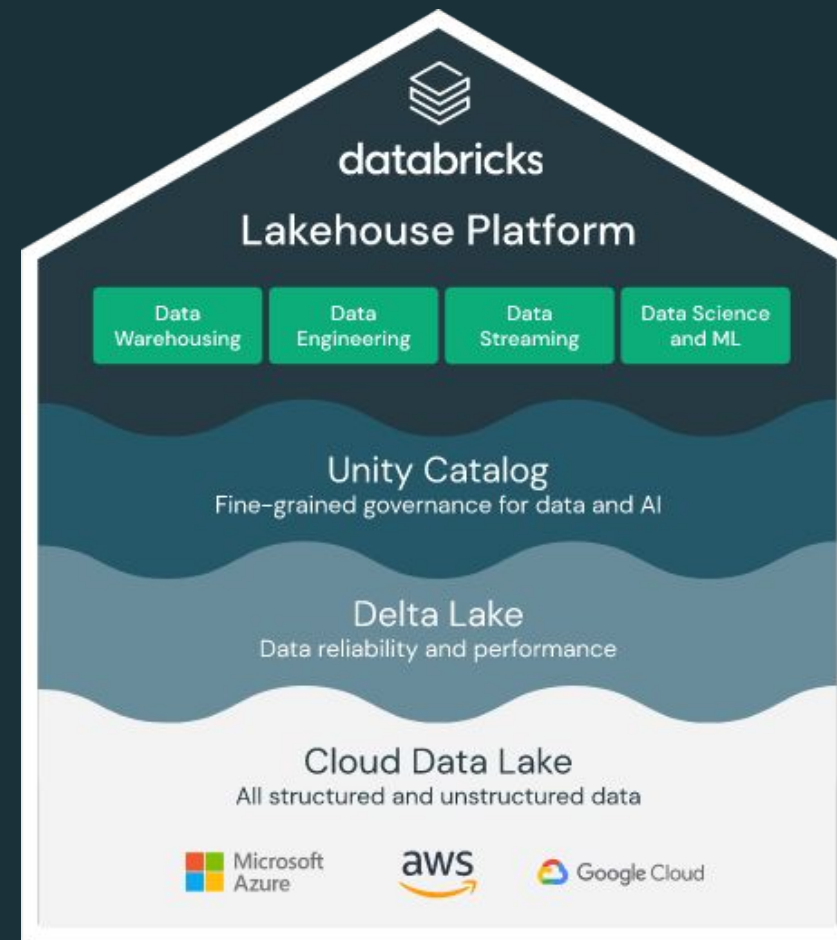
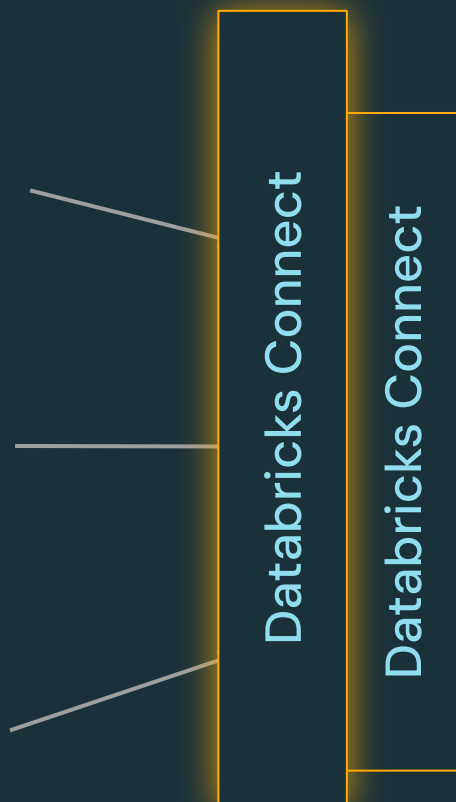
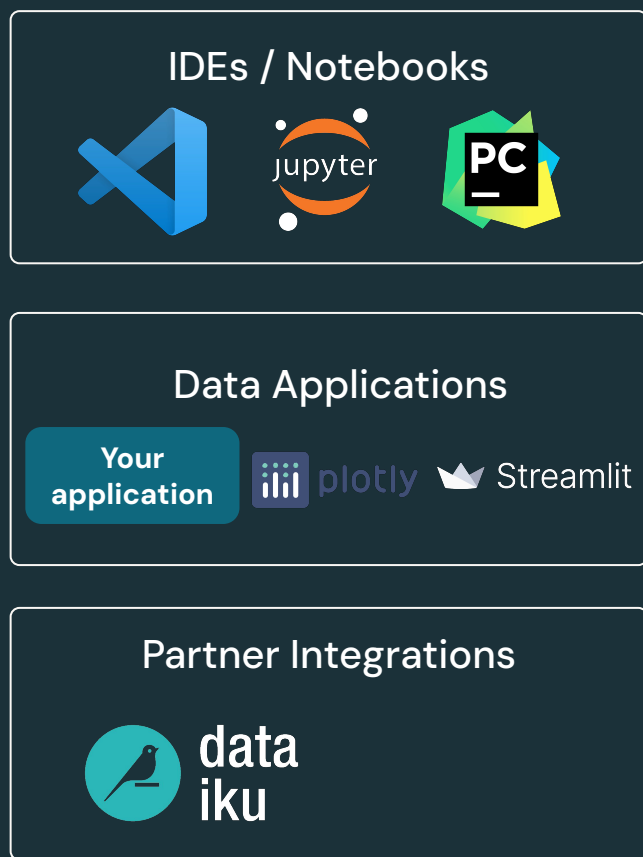
Run



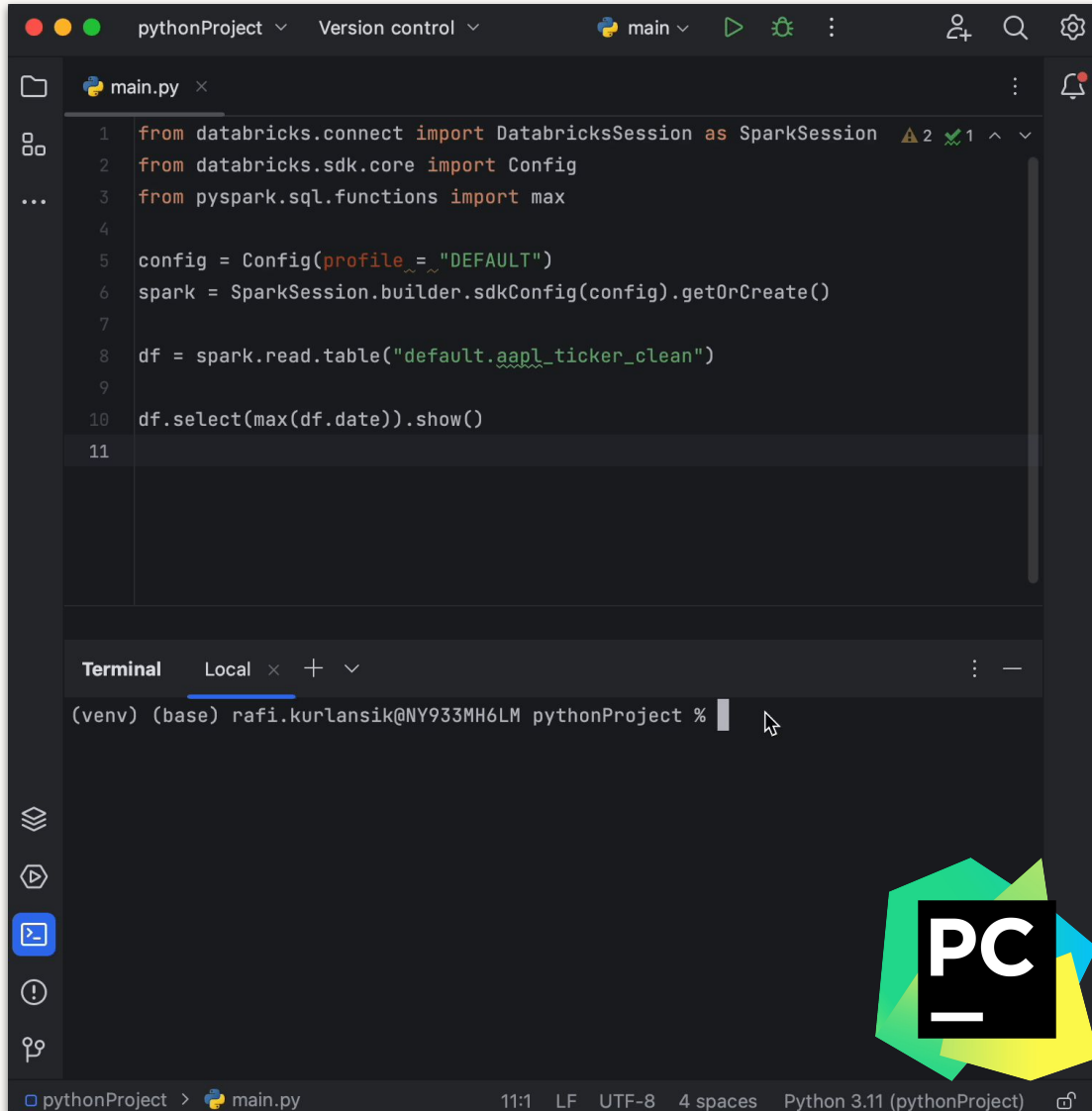
What about interactive development & debugging?

Connect to Databricks from anywhere!

Databricks Connect "v2" powered by Spark Connect



Databricks Connect 2.0



```
1 from databricks.connect import DatabricksSession as SparkSession
2 from databricks.sdk.core import Config
3 from pyspark.sql.functions import max
4
5 config = Config(profile="DEFAULT")
6 spark = SparkSession.builder.sdkConfig(config).getOrCreate()
7
8 df = spark.read.table("default.aapl_ticker_clean")
9
10 df.select(max(df.date)).show()
11
```

Terminal Local x + v

(venv) (base) rafi.kurlansik@NY933MH6LM pythonProject %

pythonProject > main.py 11:1 LF UTF-8 4 spaces Python 3.11 (pythonProject)

Redesigned to use the decoupled client-server architecture of Spark Connect, Databricks Connect allows you to run Apache Spark remotely on a Databricks cluster, instead of in the local Spark session.

- **Run large-scale Spark jobs from any Python application**, without needing to install any IDE plugins or use Spark submission scripts.
- **Step through and debug code in any IDE** even when working with a remote cluster.
- **Be more productive** – by isolating client sessions users can maintain progress if clusters restart or dependencies change



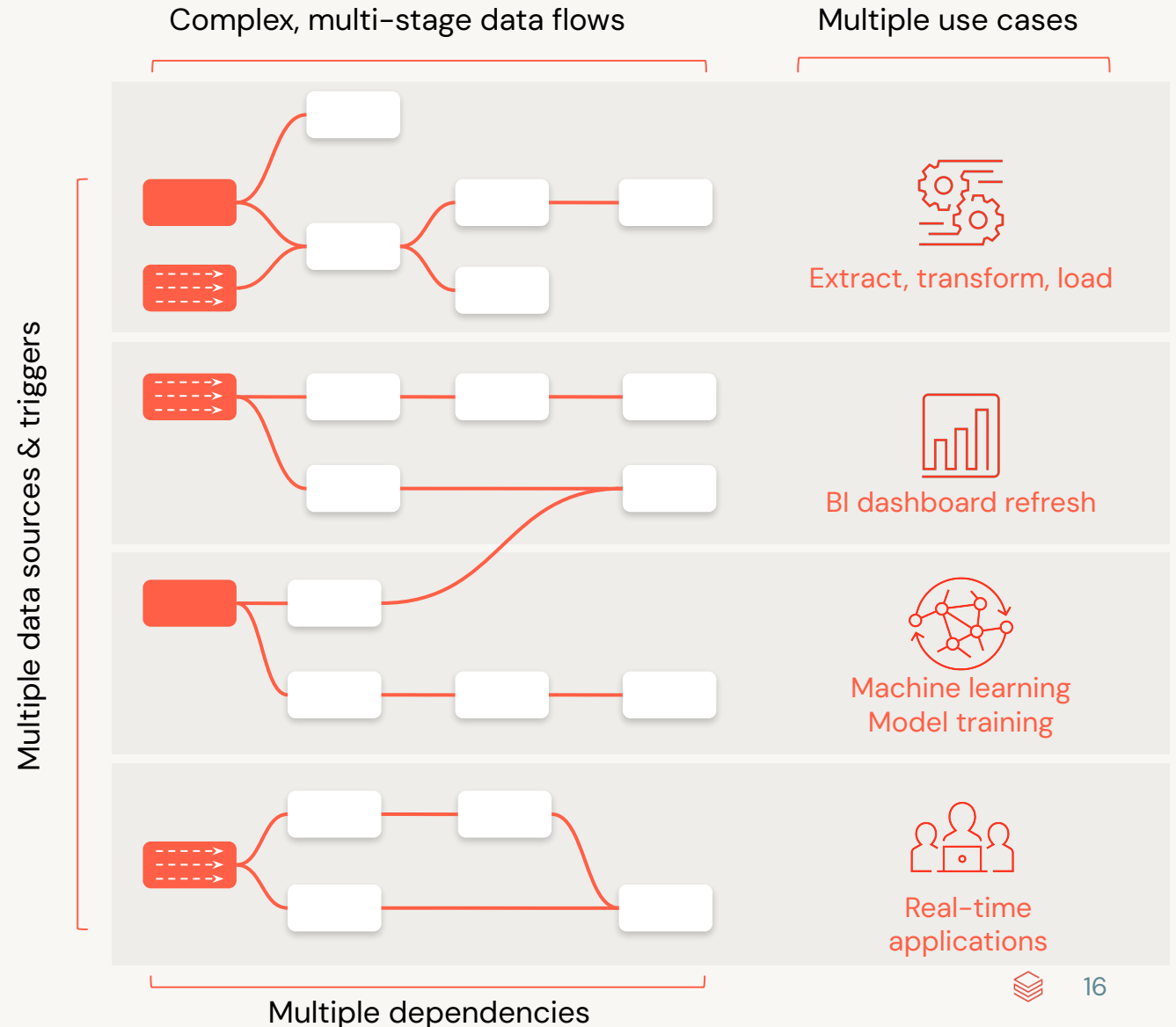
Databricks Workflows

Modern data engineering requires modern data orchestration

Orchestrating processes across all data, analytics and AI use cases is business critical

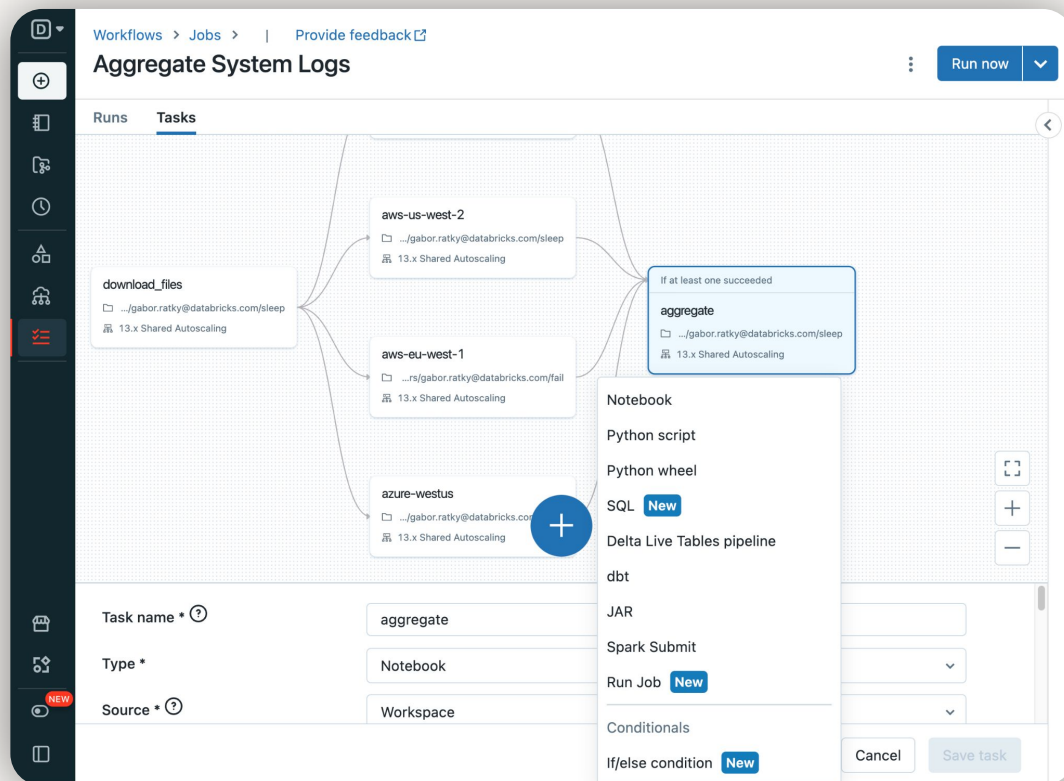
“Data pipelines are growing in size, volume, and complexity, with multistage processing and dependencies between various data assets.”*

**Gartner Data Engineering Essentials, Patterns and Best Practices, September 2022*

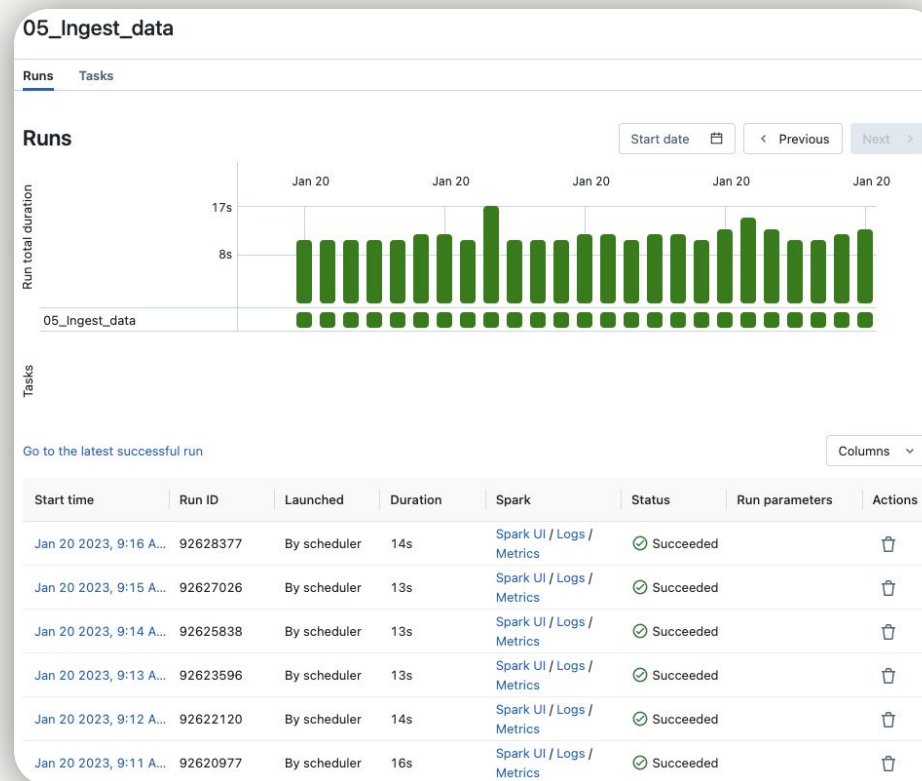


Databricks Workflows

Scheduling & orchestration made simple



Build sophisticated workflows inside your Databricks workspace with a few clicks

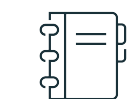


Get notifications and monitor your workflows for actionable insights

Building Blocks of Databricks Workflows

A unit of orchestration in Databricks Workflows is called a Job.

Jobs consist of one or more **Tasks**



Databricks Notebooks



Python Scripts



Python Wheels



SQL Files/Queries



DBSQL Dashboards



Delta Live Tables Pipeline



dbt



Java JAR file



Spark Submit

Control flows can be established between **Tasks**.

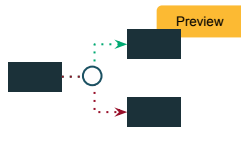
Jobs supports different **Triggers**



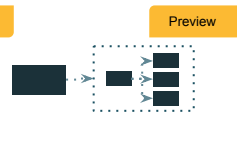
Sequential



Parallel



Conditionals (Run If)



Modular (Run Jobs)



Manual Trigger



Scheduled (Cron)



API Trigger



File Arrival



Delta Table Update



Continuous (Streaming)



Interactive Clusters



Jobs Clusters (~50% Cheaper)



Serverless Job Clusters



Job Cluster Re-Use



Repair & Re-run

And so much more!

More on Tasks

You can specify the *source* of your Task code to run from a:



Databricks Workspace
Quickstart from the File Browser



Remote Git Repository
Simplify CI/CD

You can install **external dependent libraries** such as JARs, Python Eggs and Wheels from the following sources:



Maven



CRAN



PyPI



DBFS



Workspace



Upload from your
machine!

Tasks can **trigger emails** to stakeholders enabling you to **proactively monitor and take action against late or long running jobs**.



- For example, you can define the below rules for a job that **usually takes 5 mins!**
- Send me an **warning** if my job **doesn't complete after 10 mins**
 - **Time-out** the job completely **after 15 minutes to save costs**

You can also configure **when and how many times failed runs are retried!**
(e.g *retry 5 times and wait 10 minutes between retries*)



If a *multi-task* job fails , you can **repair and re-run only the failed tasks** reducing the time & resources required to recover - **saving you time and money!**

Workflows > Jobs > | [Provide feedback](#)

Databricks Workflows

Runs Tasks

Task name * ⓘ Task

Type * Notebook

Source * ⓘ Workspace

Path * ⓘ Select Notebook

Compute * ⓘ Serverless_jobs_compute **Serverless** 2X-Small · 13-14 DBU/h ⓘ

Dependent libraries ⓘ + Add

Parameters ⓘ + Add UI | JSON

Emails ⓘ

prashanth@databricks.com ⓘ

☒ Start ☒ Success ☒ Failure ☒ Duration warning

prashanth.babu@databricks.com ⓘ

☐ Start ☐ Success ☒ Failure ☐ Duration warning

+ Add

Retries ⓘ + Add

Duration threshold Warning ⓘ Timeout ⓘ

00h 10m 00s ⓘ 00h 15m 00s ⓘ



More on Control Flows

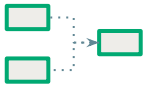
Build more **dynamic, mature, and sophisticated** data pipelines with Databricks Workflows!

Task Dependencies

When a task is **Done**, it can be in a **Success**, **Failure**, or **Excluded** state.

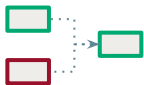
All Succeeded

Default behaviour



At Least 1 Succeeded

e.g. Fan in with at least some success



None Failed

e.g. Run task(s) at the end of DAG if nothing fails



All Done

e.g. Perform clean up even if tasks have failed or excluded



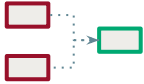
All Least 1 Failed

e.g. Perform clean-up with observability or specific actions

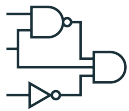


All Failed

e.g. Perform clean-up with observability or specific actions



Parameterisation



Job Parameters

Passed into each Task with behaviour based on the type
e.g. additional options for JARs, spark-submit, Python Args



Job Contexts

Special set of templated variables that provide introspective metadata about job and task
e.g. run_id, job_id, start_time



Task Values

Custom parameters that can be shared between Tasks in a Job
e.g. anything that can be programmatically set or retrieved!

Webhooks

Allows customers to build **event-driven integrations** with Databricks.

Supported destinations are **Slack** and **Webhooks**, with the below **notification events**:

For example, you can send a message to a Slack #channel when:



On start: Send a message to a when a job or a parent run is started



On success: when a job or a parent run finished without any errors



On failure: when a job fails or a parent run is terminated with one of the children in a failed state.



Enhanced control flows

Adding enhanced control flow logic to workflows

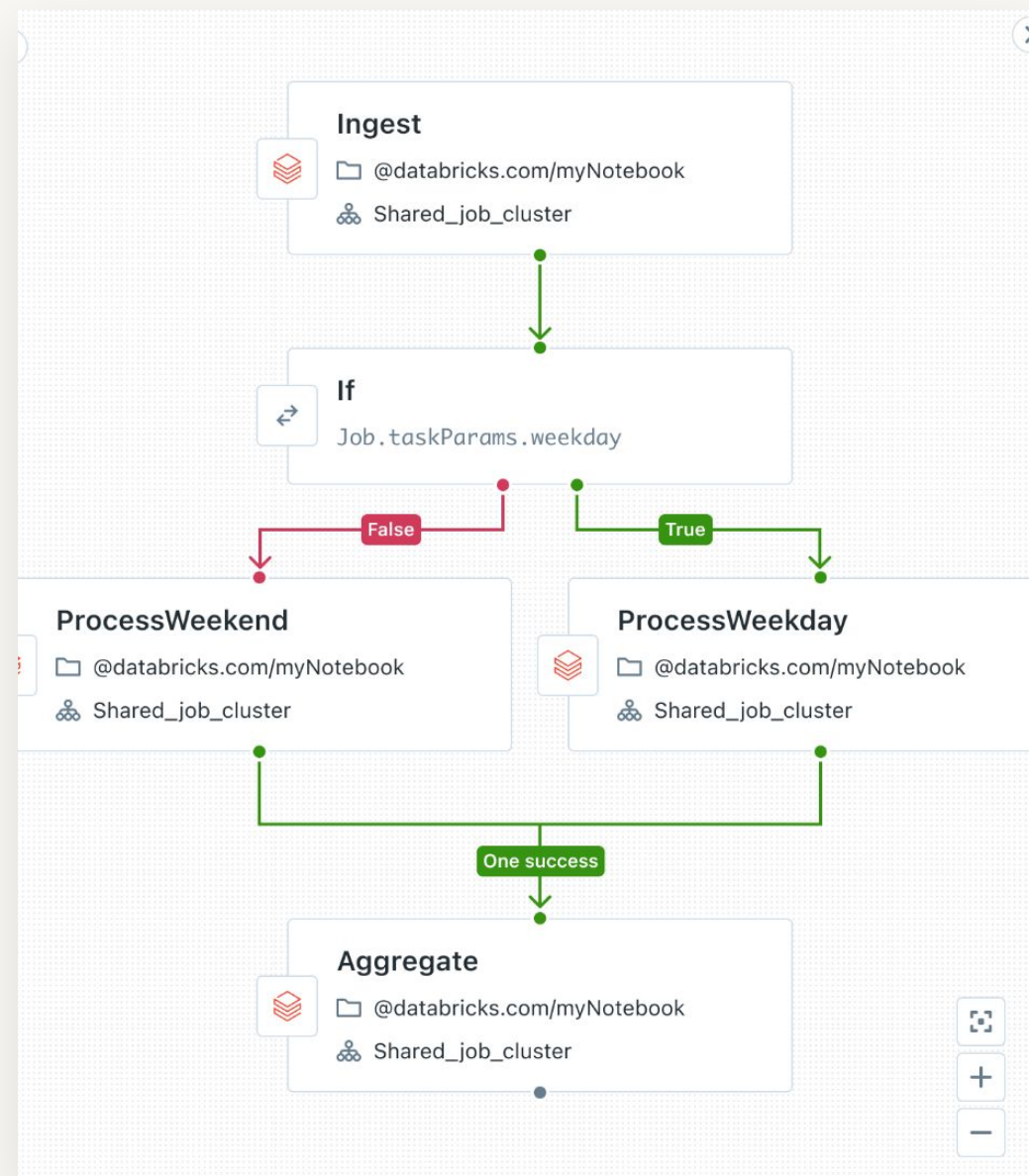
Define sophisticated multi-task workflow logic that executes different tasks according to runtime conditions

Incorporate error handling in workflow definitions

Exclude unnecessary tasks during runtime for higher efficiency

Full visibility to what tasks executed in every run

GA in Q2



Workflows as tasks

Run a workflow as a task in another workflow

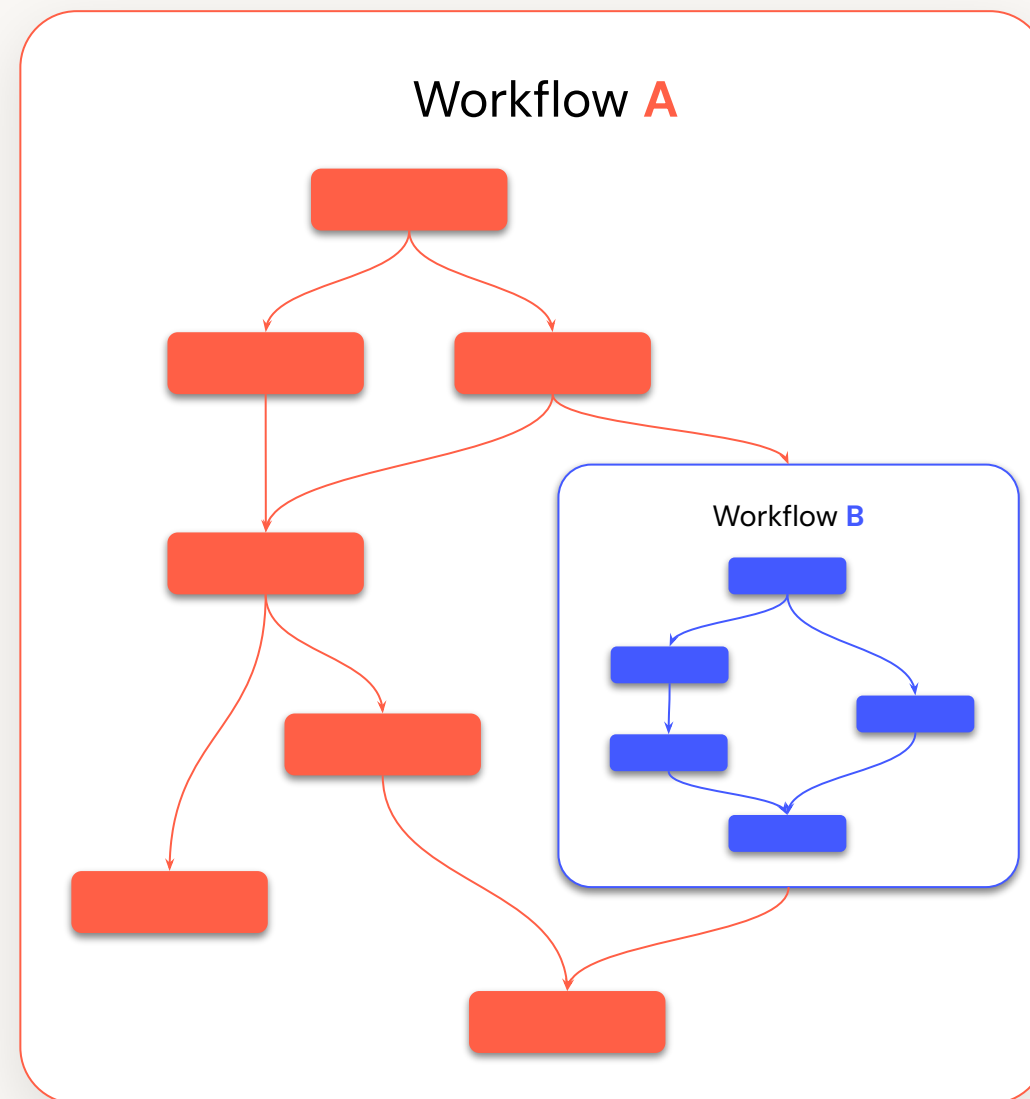
Break down complex DAGs into smaller, repeatable units of execution that can be triggered as tasks

Define tasks that trigger other workflows

Simplify authoring, testing and troubleshooting

Reuse workflow definitions to save time

GA in Q2



Filesystems on Databricks

Overview Filesystems

Workspace File System

- Access controlled via workspace permissions
- Use for storing code (notebooks, scripts, files) with and w/o Repos
- Physically located on Databricks Control Plane

Access from Spark

```
dbutils.fs.ls(file:/Workspace/Users/)  
dbutils.fs.ls(file:/Workspace/Repos/)
```

Access from Driver/Worker (POSIX)

```
os.listdir(/Workspace/Users/)  
os.listdir(/Workspace/Repos/)
```

Databricks File Systems

- Not access controlled
- Used to store unclassified data and add legacy mount point.
- Physically location on customers root object storage
- Not recommended for production use if using UC

Access from Spark

```
dbutils.fs.ls(dbfs:)  
dbutils.fs.ls(/)
```

Access from Driver/Worker (POSIX)

```
os.listdir(/dbfs/)
```

Unity Catalog Volumes

- Accessed controlled via Unity Catalog
- Used to store unstructured data and access external data on object storage
- Physically located on customers selected object storage

Access from Spark

```
dbutils.fs.ls(/Volumes/<catalog>/<schema>/<volume>)
```

Access from Driver/Worker (POSIX)

```
os.listdir(/Volumes/<catalog>/<schema>/<volume>)
```

Databricks Repos

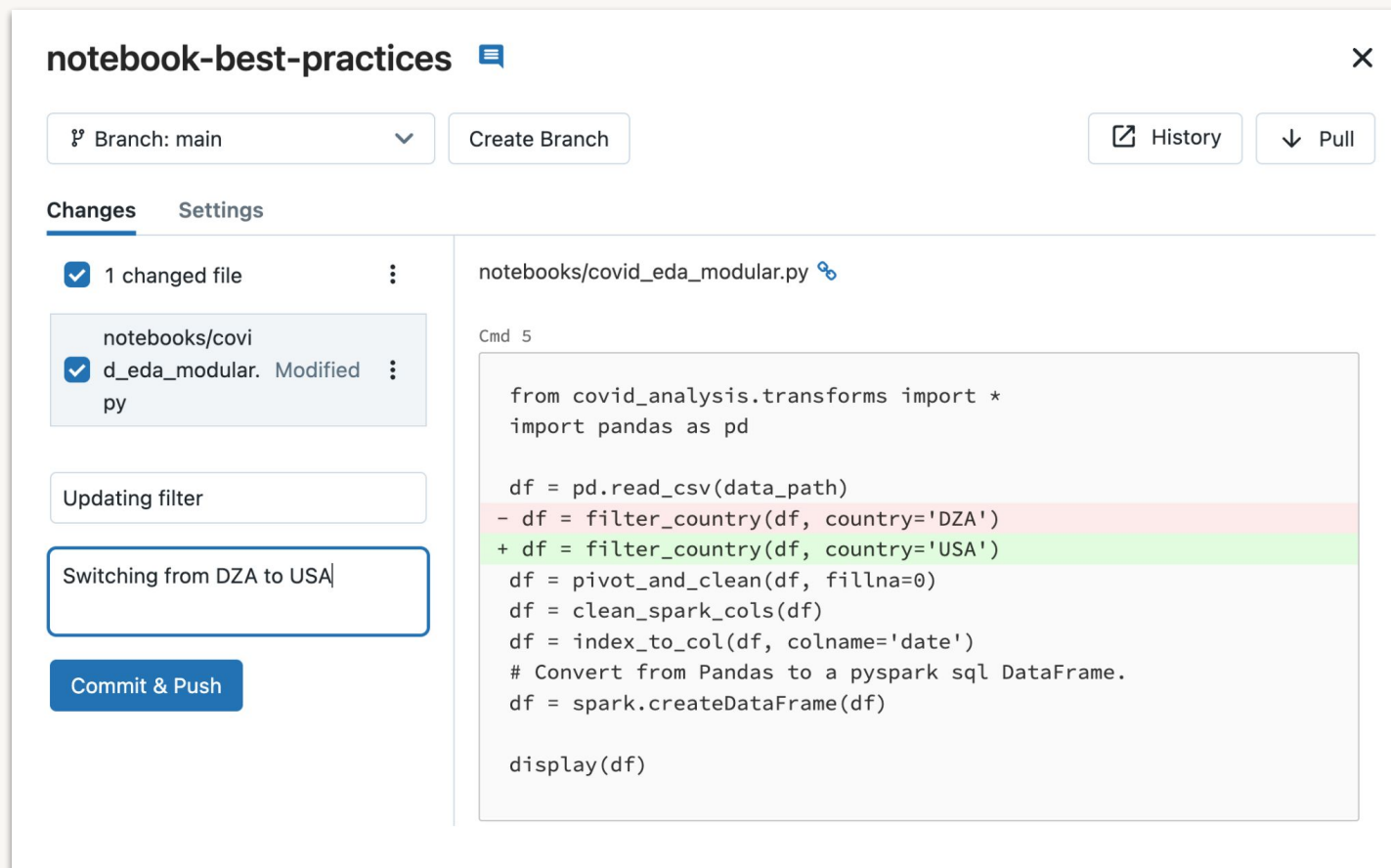
Integrate git repositories into workspace filesystem

Databricks Repos support for git lets users collaborate on the same code without interfering with each others' work.

Inspect, commit, push and pull code changes. Create and work with different branches.

New – Resolve merge conflicts.

Protected branches and pull requests provide guardrails for what code moves to production



The screenshot shows the Databricks Repos interface for a notebook named 'notebook-best-practices'. At the top, there's a dropdown for 'Branch: main' and a 'Create Branch' button. To the right are 'History' and 'Pull' buttons. Below this, there are tabs for 'Changes' and 'Settings'. Under 'Changes', it shows '1 changed file' and a list of changes: 'notebooks/covid_eda_modular.py' with a checkmark and 'Modified'. Below the list is an 'Updating filter' input field and a text box containing 'Switching from DZA to USA'. A 'Commit & Push' button is at the bottom left. On the right, the notebook content is displayed, showing a Python script. The script imports 'pandas' and uses 'filter_country' to filter data by country. The line 'df = filter_country(df, country='DZA')' is highlighted in red, and the line '+ df = filter_country(df, country='USA')' is highlighted in green, indicating the change being made. The script also includes functions for pivoting, cleaning, and converting the data to a PySpark DataFrame.

Volumes

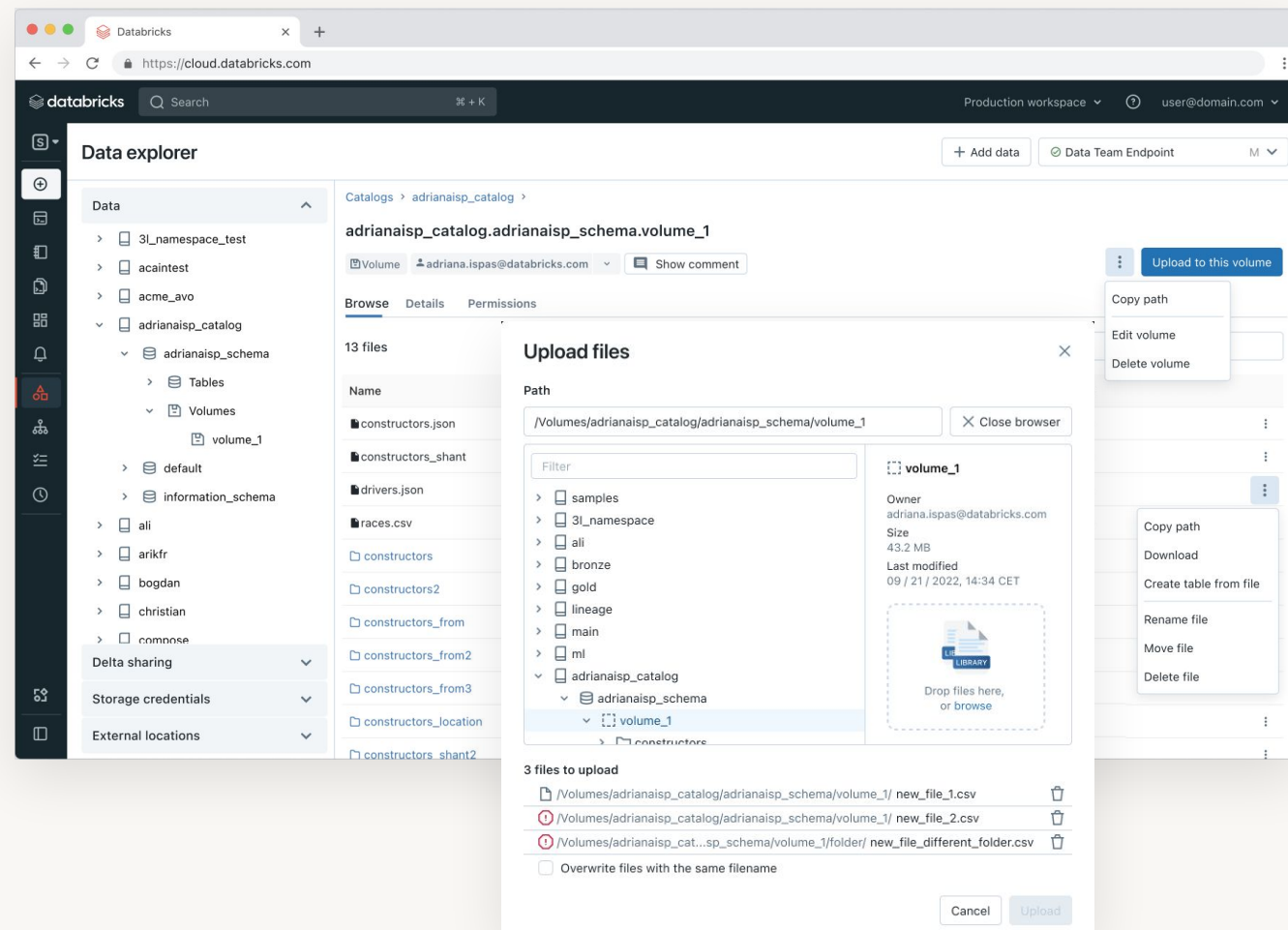
Catalog collections of files in Unity Catalog

Access, store, organize and process **non-tabular data** with Unity Catalog governance

Unlock new processing capabilities for arbitrary files, including data science and machine learning

Any file format; data can be structured, semi-structured, or unstructured

Files accessible via UI, Spark APIs, FUSE, dbutils, REST, SQL, Databricks CLI, Terraform.



Developer Tools

Fully automate your Lakehouse operations

Databricks REST API

- Generally Available
- Full platform automation!
- docs.databricks.com/api/workspace/introduction



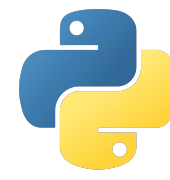
Databricks CLI

- Public Preview
- Follows OpenAPI spec
- docs.databricks.com/dev-tools/cli/databricks-cli



Databricks SDKs

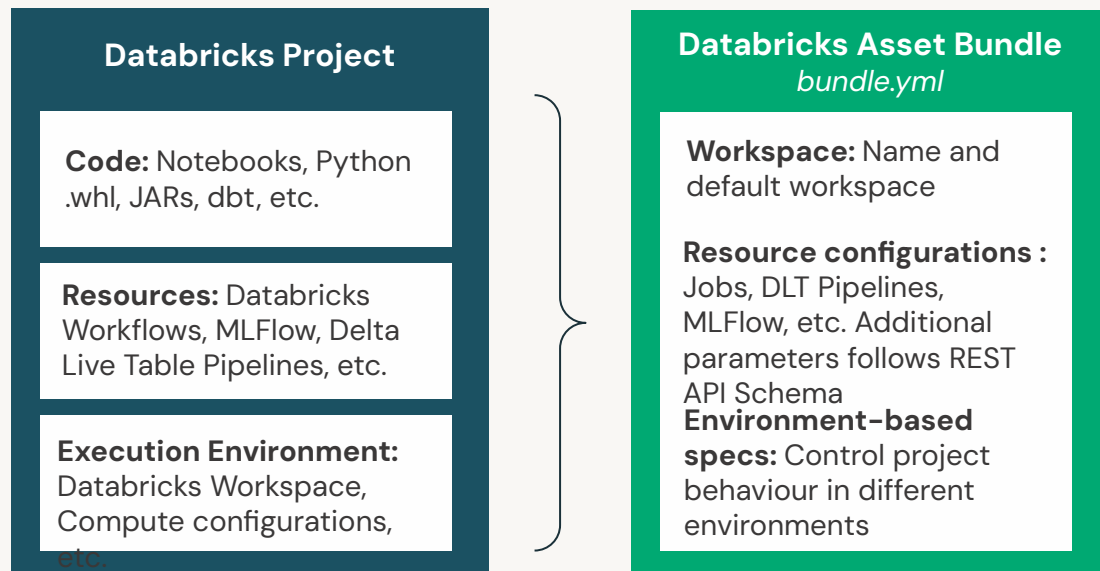
- Python, Go, Java SDKs following OpenAPI spec
- Loads of examples to try!
- docs.databricks.com/dev-tools/sdk-python



Databricks Asset Bundles

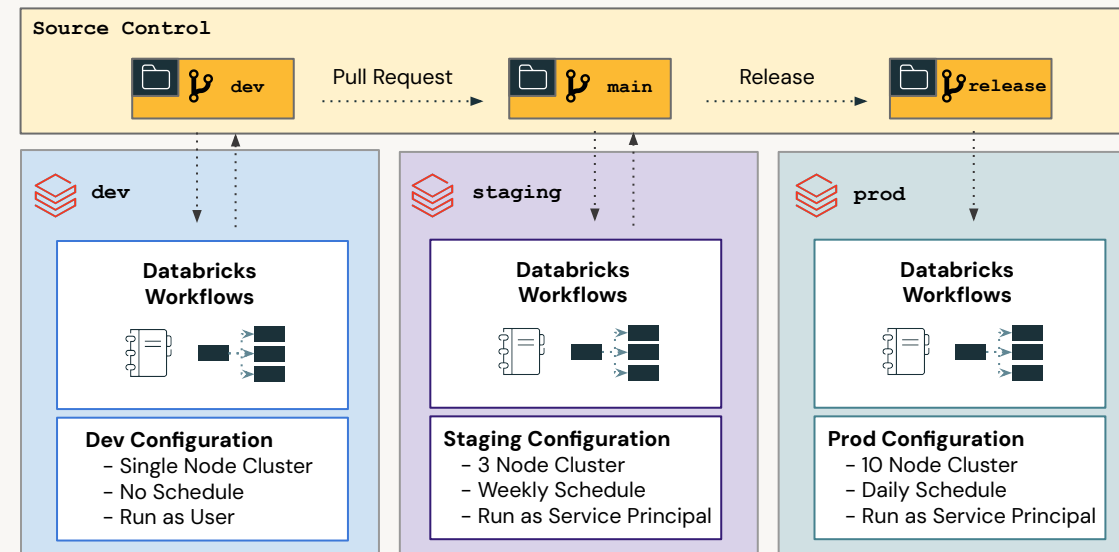
What are Databricks Asset Bundles

Databricks Asset Bundles (DABs) are YAML files that specify the artifacts, resources and configurations of a Databricks project.



You can deploy to multiple workspaces, run your projects, tweak configs, and test changes from your IDEs, Terminal or from within Databricks!

You can **install the Databricks CLI** and begin to **deploy your bundles on your CI/CD server**; you can also **trigger automatically with Git Actions**



```
$ databricks bundle deploy -t "dev"
$ databricks bundle run pipeline --refresh-all -t "dev"
```

Alternative Options



Python SDK

The Jobs Python API allows you to create, edit, and delete jobs



Terraform Provider

The databricks_jobs resource can be used to create and manage Jobs with Terraform



Databricks REST API

Low-level REST API allows you to manage Jobs programmatically



Bundle definition

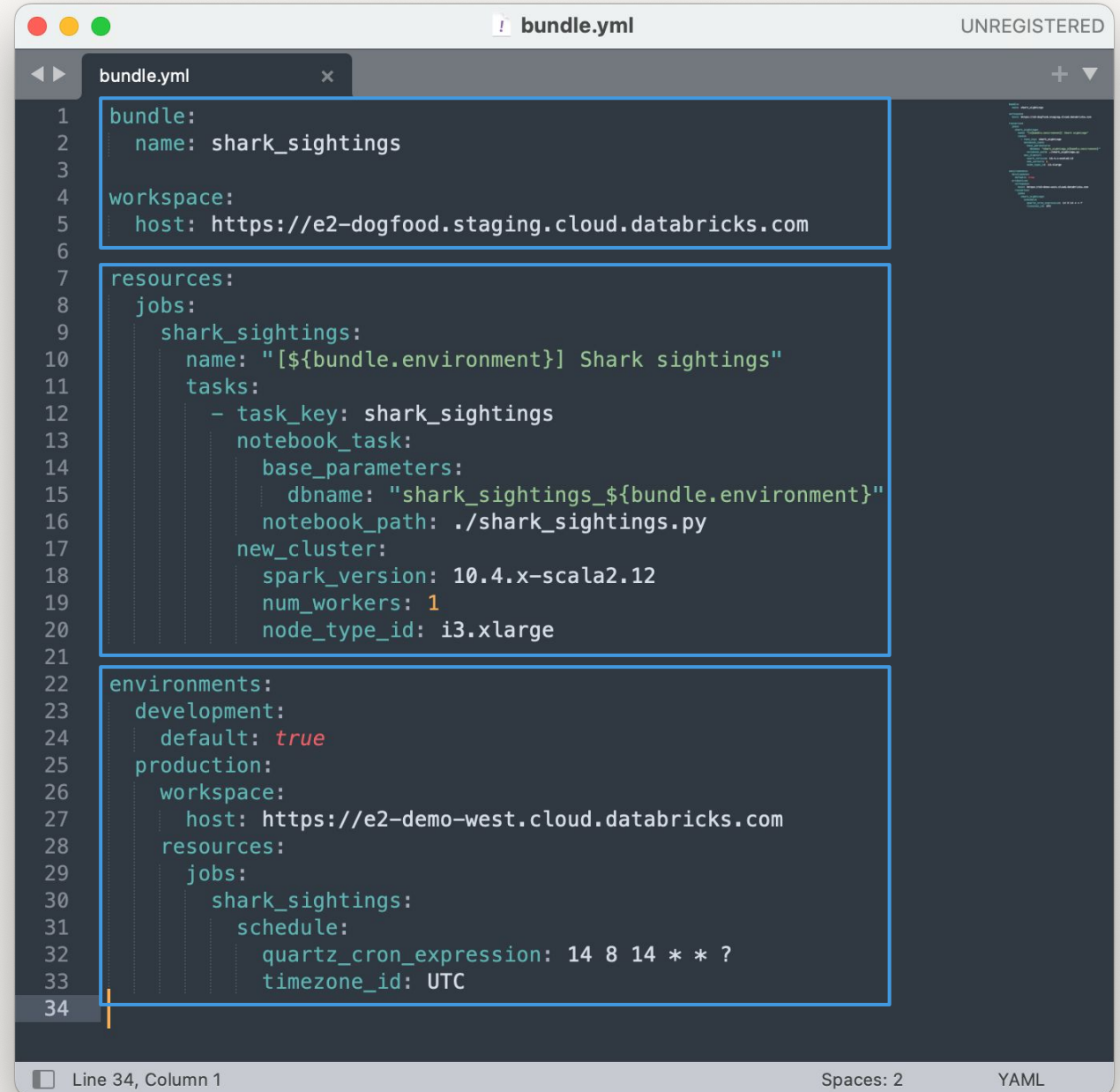
Name and default Workspace

Resource configurations

- Jobs, DLT pipelines, MLflow, etc.
- Follows REST API schema

Environment-based specs

- Control project behavior in different environments
- Set variables or override default configs for environment based configuration



```
bundle.yml
UNREGISTERED

1 bundle:
2   name: shark_sightings
3
4 workspace:
5   host: https://e2-dogfood.staging.cloud.databricks.com
6
7 resources:
8   jobs:
9     shark_sightings:
10      name: "[${bundle.environment}] Shark sightings"
11      tasks:
12        - task_key: shark_sightings
13          notebook_task:
14            base_parameters:
15              dbname: "shark_sightings_${bundle.environment}"
16            notebook_path: ./shark_sightings.py
17          new_cluster:
18            spark_version: 10.4.x-scala2.12
19            num_workers: 1
20            node_type_id: i3.xlarge
21
22 environments:
23   development:
24     default: true
25   production:
26     workspace:
27       host: https://e2-demo-west.cloud.databricks.com
28     resources:
29       jobs:
30         shark_sightings:
31           schedule:
32             quartz_cron_expression: 14 8 14 * * ?
33             timezone_id: UTC
34
```

Other Bundle feature

- **Templates:** Use build-in or custom templates to initialize a complete project structure
- **Idempotent Operations:** Apply the same configuration multiple times, changes are only made when there are differences between the desired and current state. State itself is managed automatically on workspace fs.
- **User separation on workspace:** Multiple users can deploy the same project without conflicts. Code is separated by directory on workspace fs, other resources by Owner.
- **Fixed root path:** For production deployment a fixed workspace fs path can be defined, irrespective of the user
- **Development mode:** Automatically deactivate job schedule, optimally attach job to all purpose cluster and set DLT to “development” in development environment

Summary

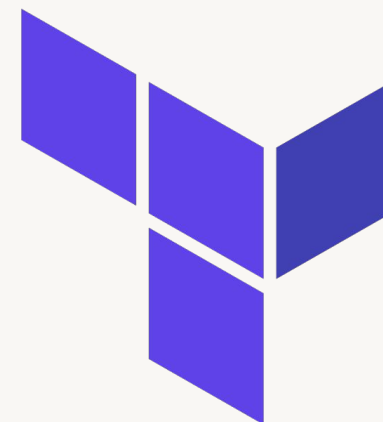
- DABs are part of the new CLI
- They simplify your CI/CD setup providing features that you would otherwise need to build yourself from with the other dev tools
- They are the recommended way to do CI/CD on Databricks

Databricks Terraform Provider

Declarative Infrastructure with Terraform

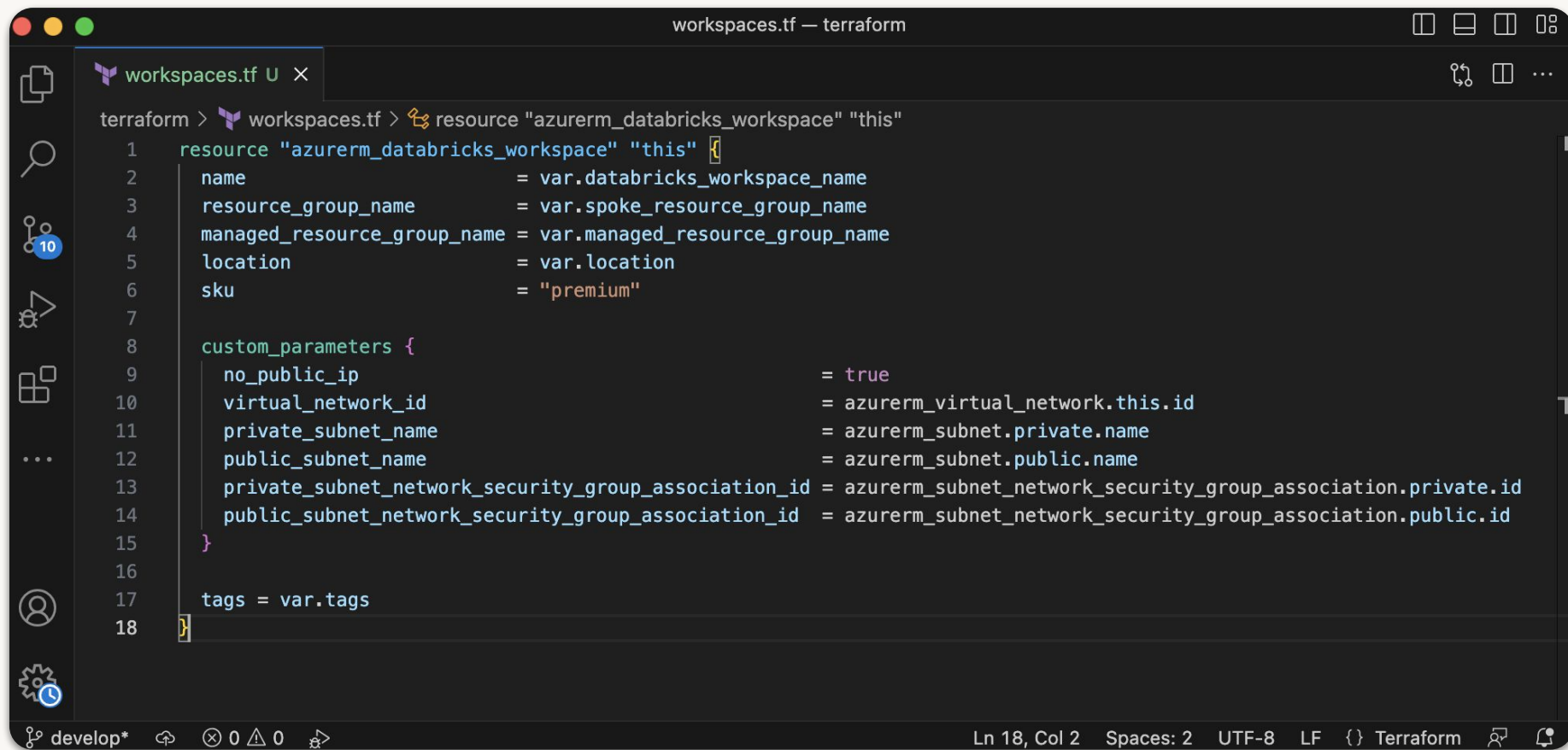
Declaratively define what cloud infrastructure is needed

- Keeps track of infrastructure state
- Automatic handling of dependencies between objects
- One command create/update/delete of all objects
- Easy creation and management of different environments
- TF Input Variables to parametrize deployments / reduce repetition
- Use Terraform Exporter to export a workflow defined in the UI to HCL Terraform definition
- Modularize code
- Open source



Using Terraform with Databricks

Deploy Databricks Workspaces and related infrastructure



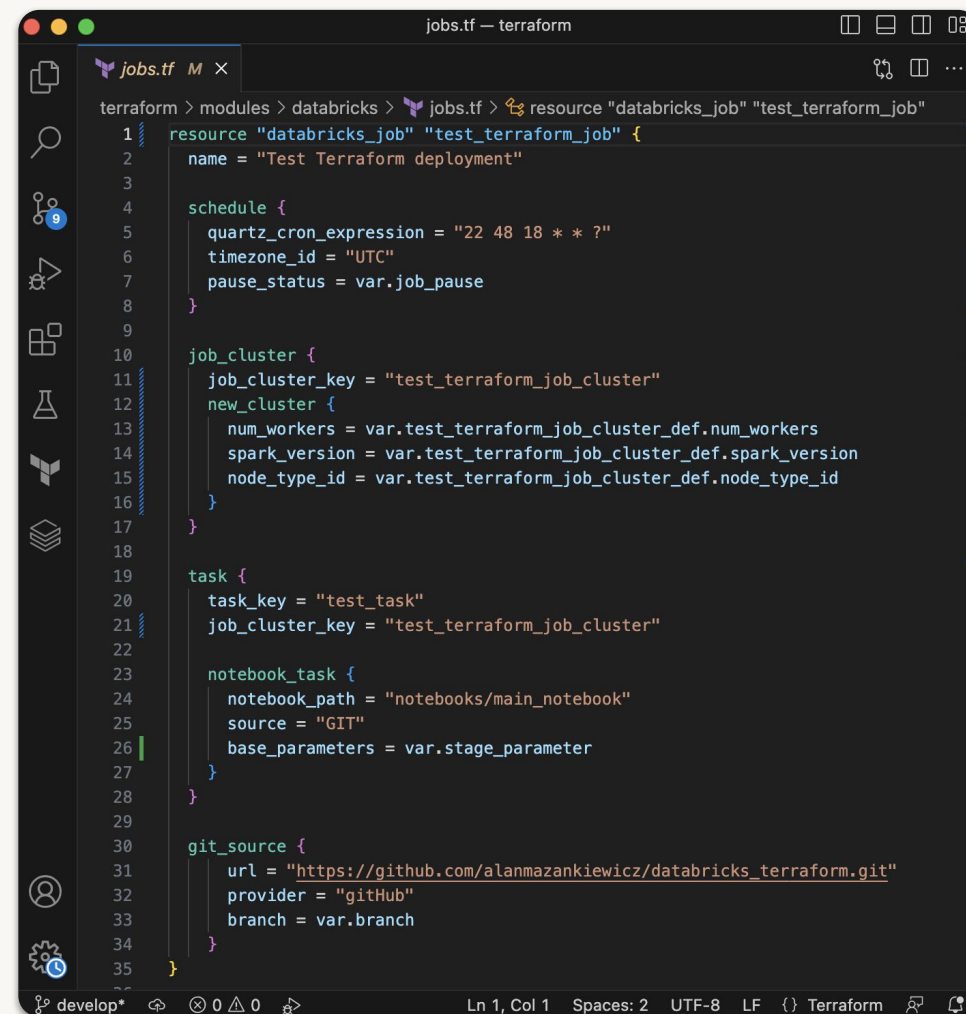
```
workspaces.tf — terraform

terraform > workspaces.tf > resource "azurerm_databricks_workspace" "this"
1  resource "azurerm_databricks_workspace" "this" {
2      name                        = var.databricks_workspace_name
3      resource_group_name        = var.spoke_resource_group_name
4      managed_resource_group_name = var.managed_resource_group_name
5      location                   = var.location
6      sku                        = "premium"
7
8      custom_parameters {
9          no_public_ip                = true
10         virtual_network_id          = azurerm_virtual_network.this.id
11         private_subnet_name         = azurerm_subnet.private.name
12         public_subnet_name          = azurerm_subnet.public.name
13         private_subnet_network_security_group_association_id = azurerm_subnet_network_security_group_association.private.id
14         public_subnet_network_security_group_association_id = azurerm_subnet_network_security_group_association.public.id
15     }
16
17     tags = var.tags
18 }
```

Databricks Terraform Provider

Define resources in the Databricks workspace

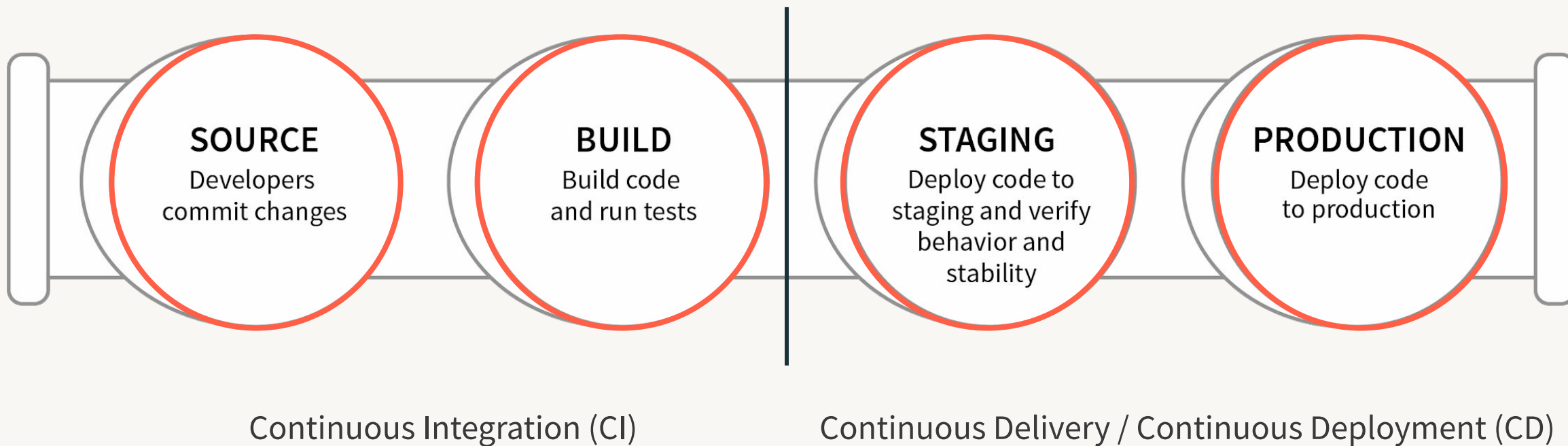
- Authentication using Databricks Unified Authentication
- Supports all objects that are exposed via Databricks REST API (clusters, jobs, users/groups, notebooks, ...)
- Alternative to DABs for project deployment
- Fully supported part of the platform



```
jobs.tf — terraform
jobs.tf M X
terraform > modules > databricks > jobs.tf > resource "databricks_job" "test_terraform_job"
1 resource "databricks_job" "test_terraform_job" {
2   name = "Test Terraform deployment"
3
4   schedule {
5     quartz_cron_expression = "22 48 18 * * ?"
6     timezone_id = "UTC"
7     pause_status = var.job_pause
8   }
9
10  job_cluster {
11    job_cluster_key = "test_terraform_job_cluster"
12    new_cluster {
13      num_workers = var.test_terraform_job_cluster_def.num_workers
14      spark_version = var.test_terraform_job_cluster_def.spark_version
15      node_type_id = var.test_terraform_job_cluster_def.node_type_id
16    }
17  }
18
19  task {
20    task_key = "test_task"
21    job_cluster_key = "test_terraform_job_cluster"
22
23    notebook_task {
24      notebook_path = "notebooks/main_notebook"
25      source = "GIT"
26      base_parameters = var.stage_parameter
27    }
28  }
29
30  git_source {
31    url = "https://github.com/alanmazankiewicz/databricks_terraform.git"
32    provider = "gitHub"
33    branch = var.branch
34  }
35 }
```

What is CI/CD?

Fast and Secure Software Development



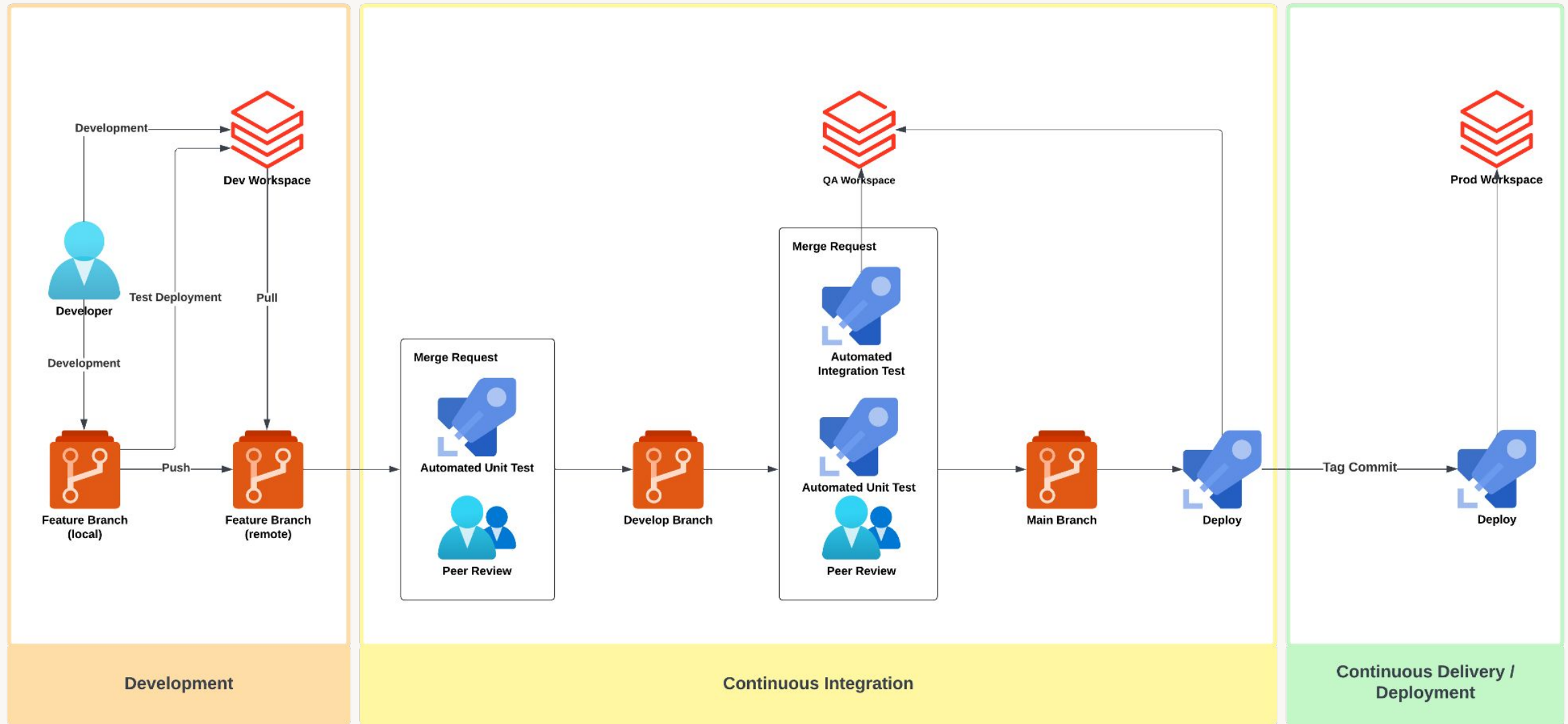
- **Quality control process** before releasing → capture errors early
- **High degree of automation** in that processes → deliver changes quickly and roll back easily

CI/CD Principles

- All artefacts that influence the behaviour of the system are **versioned** (code, configurations, data, models) → Easy rollback on error and traceability
- Code is **deployed** to production through an **automated process** without manual steps → Fast, repeatable deployment and easy rollback
- All code changes go through an **quality control process**
 - Merge requests with **automated tests** and manual code review
- Production, testing and development environments are **isolated** and access controlled
- Production environments are **monitored** to ensure correct behaviour
- Code is **modularized** to make it easy to understand, test and minimise repetition



Abstract CI/CD workflow on Databricks



Notes

- Within a workspace workflows are separated by owner (user) and code (Workspace FS / Repos) by user directory
 - Each user should create their own connection to the git repo in their user directory to separate code development in dev
 - Using dev tools (e.g. CLI) users can make test deployments from local env to their user dir in the dev workspace
- Run fast unit tests before slower integration tests
- Changes to develop branch during merge request to main branch should invalidate testing pipeline runs and reviewer approvals
- QA is used to (1) run integration tests triggered from CI pipeline during merge request and (2) do a pre-production deployment (pipelines run on regular schedule) to monitor pipelines over some time (by devs or selected downstream stakeholder). Separate (1) and (2) with different Service Principals.
- Prod and QA should be read only. Prod can sometime even be restricted to privileged users only (e.g. when data is sensitive).
- Prod environment must only have access to Prod data (e.g. via workspace restricted catalog, access control for service principal). If prod data not sensitive, dev environment can have (read only) access to all data (e.g. prod, dev).



Project Structure

Common patterns



- **Notebooks only:**
 - Step by step debugging, faster feedback, improved monitoring of runs
 - Harder to automate, challenges with testing and modularisation
- **Artifact only:**
 - Develop Python / Scala code in IDE, build, execute as jobs
 - Better automation, more tooling, delayed feedback loop
- **Mixed (typical usage by our customers):**
 - Code developed in IDE (notebooks + modules to import from)
 - Notebooks are used as point of entry for configuration & orchestration of calls to modules and interactive testing during development in Databricks UI



Example Project Structure

Deployment

Stage Configuration

Source Code

Tests

Dependencies

Terraform + Azure DevOps

```
├─ azure-pipelines/  
|   ├── deploy.yml  
|   ├── integration-test.yml  
|   └─ unit-test.yml  
├─ terraform/
```

```
├─ conf/  
|   ├── prod.yml  
|   ├── dev.yml  
|   └─ integration-test.yml
```

```
├─ notebooks/  
|   ├── prepare.py  
|   ├── normalize_customers.py  
├─ customer_unification/  
|   ├── load_config.py  
|   └─ cleaning.py
```

```
├─ tests/  
|   ├── unit/  
|   └─ integration
```

```
├─ requirements.txt  
└─ test-requirements.txt
```

With DABs

```
├─ azure-pipelines/  
|   ├── deploy.yml  
|   ├── integration-test.yml  
|   └─ unit-test.yml  
├─ resources/  
|   ├── job.yml  
|   └─ integration-test.yml  
└─ databricks.yml
```

```
├─ conf/  
|   ├── prod.yml  
|   ├── dev.yml  
|   └─ integration-test.yml
```

```
├─ notebooks/  
|   ├── prepare.py  
|   ├── normalize_customers.py  
├─ customer_unification/  
|   ├── load_config.py  
|   └─ cleaning.py
```

```
├─ tests/  
|   ├── unit/  
|   └─ integration
```

```
├─ requirements.txt  
└─ test-requirements.txt
```



Stage specific configuration

- We usually need to run our code slightly differently in different stages. For example during development or integration testing:
 - We need faster iteration cycles
 - We do not want our pipeline to run on a schedule
 - We are using static subsets of our live data
 - We are using smaller clusters
- Make your application configurable through
 - Stage specific **workflow definitions** (e.g. for schedule and cluster definitions)
 - Flexible code structure (functions, classes) and stage **specific configuration file** to read parameters from (e.g. for input data locations, hyperparameter search trials, dates to filter by during integration testing)



Configure stages in Databricks Asset Bundles

- In DABs one defines stage specific configurations
 - Overwrite job specific configurations such as cluster size in dev and prod (see next slide)
 - Target workspace
 - Mode: development or production
- Development Mode:
 - Pauses all schedules and triggers on deployed jobs
 - Attaches jobs to all purpose cluster if compute id is specified
 - See [Docs](#) for more details

```
1 bundle:
2   name: my_project
3
4 targets:
5   dev:
6     mode: development
7     default: true
8     workspace:
9       host: https://dev_workspace.cloud.databricks.com
10      compute_id: my_compute_id
11
12   prod:
13     mode: production
14     workspace:
15       host: https://prod_workspace.cloud.databricks.com
16       root_path: /Shared/.bundle/prod/${bundle.name}
```



Overwrite job specific definitions in DABs

```
1 resources:
2 jobs:
3   my_project_job:
4     name: my_project_job
5     [...]
6     job_clusters:
7       - job_cluster_key: job_cluster
8         new_cluster:
9           spark_version: 13.3.x-scala2.12
10           node_type_id: i3.xlarge
11           autoscale:
12             min_workers: 1
13             max_workers: 2
```

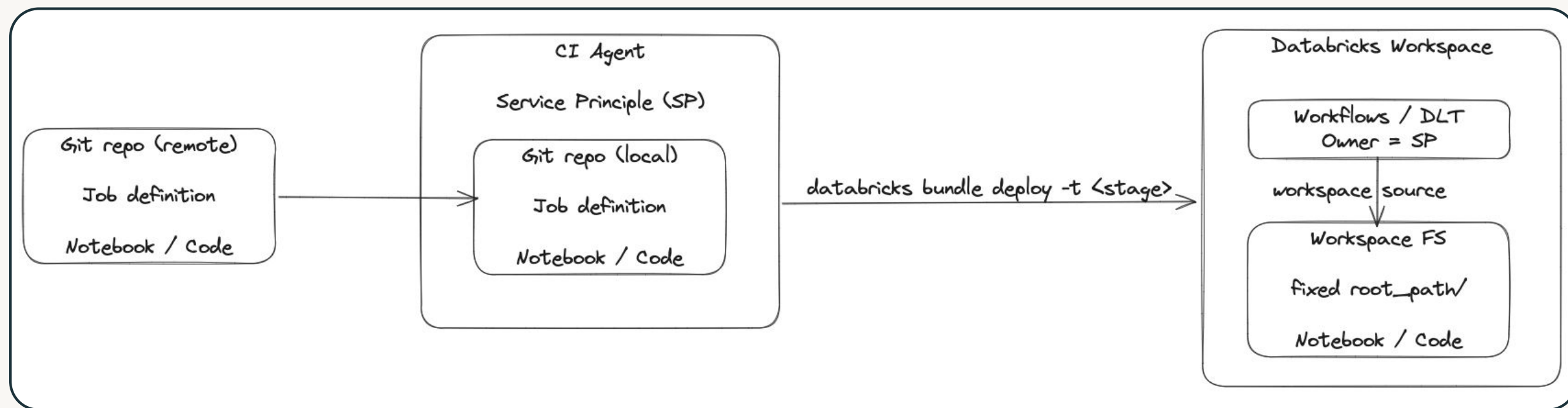
```
18 targets:
19   prod:
20     resources:
21       jobs:
22         my_project_job:
23           job_clusters:
24             - job_cluster_key: job_cluster
25               new_cluster:
26                 spark_version: 13.3.x-scala2.12
27                 node_type_id: i3.2xlarge
28                 autoscale:
29                   min_workers: 4
30                   max_workers: 8
```

- [...] – Other job specific definitions that stay the same between stages e.g. task definition, schedule etc.
- In this example no of autoscaling workers and node type gets increased in prod

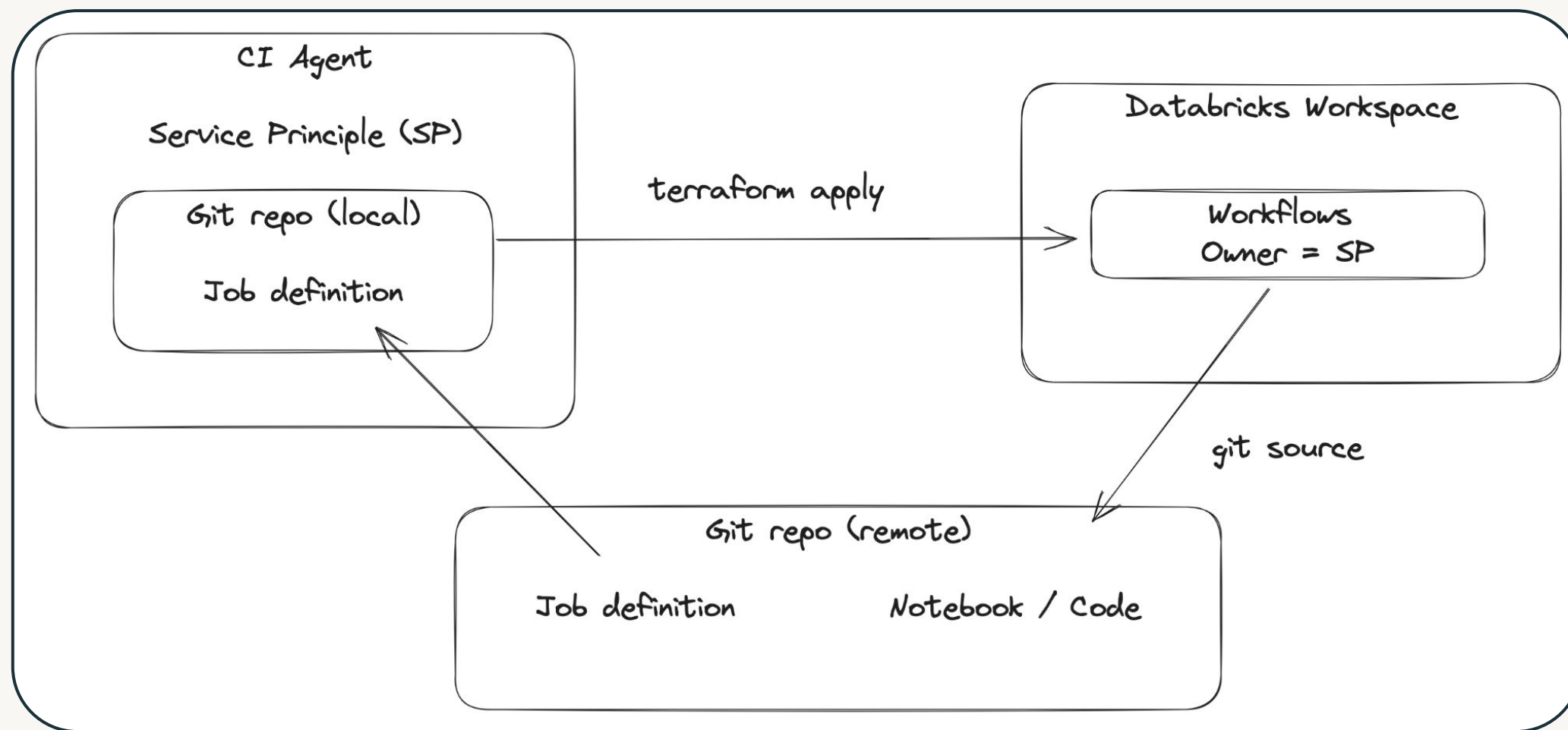


Deployment

Deployment Pattern: DABs



Deployment Pattern: Terraform and Repos



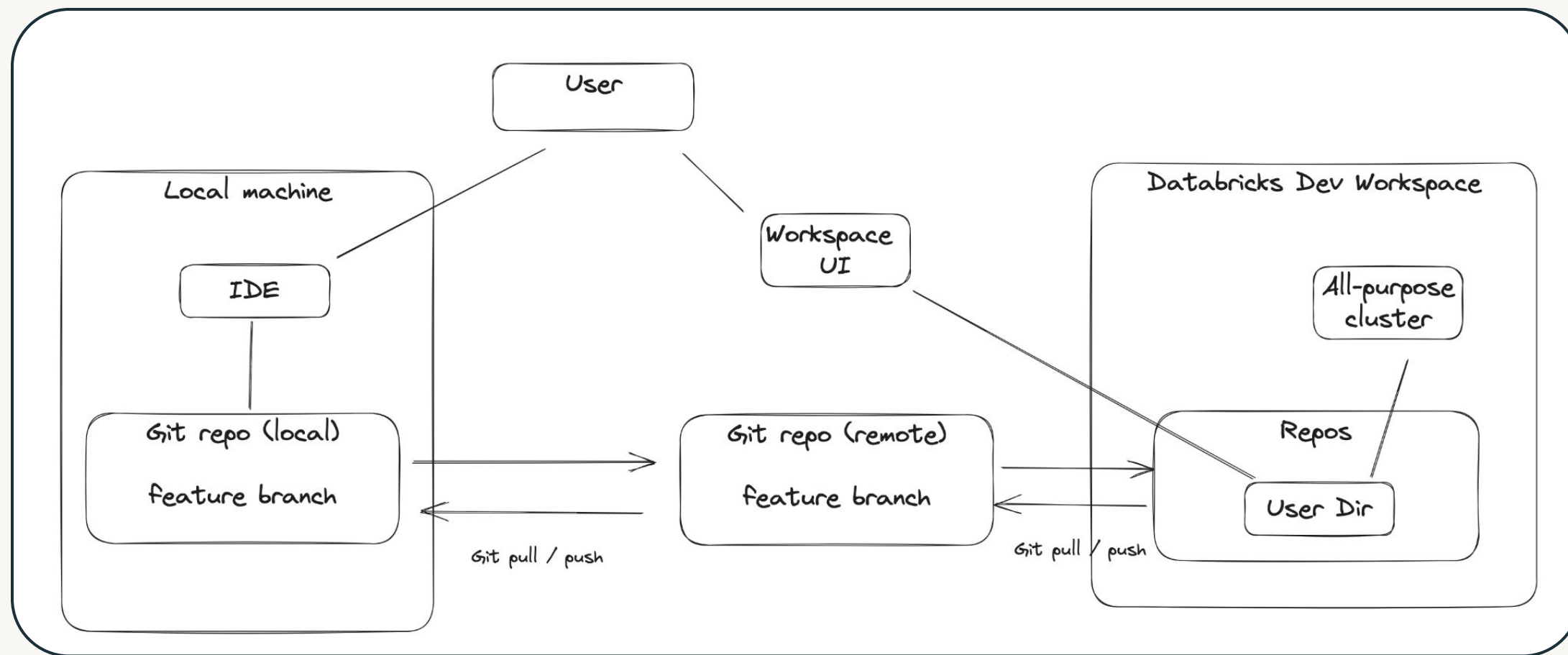
Best practice via Databricks Terraform Provider

- Notes:
 - Requires git connection to workspace
 - Requires terraform setup (manage backend, TF skills, ...)
 - Does not work with DLT since DLT does not support Git source
- Possible other options:
 - Deploy notebooks + dependencies to Workspace FS using [databricks_sync](#)
 - Deploy notebooks using TF to workspace
 - Deploy job definition via CLI / SDK



Development workflows

Development Pattern: IDE and UI Hybrid

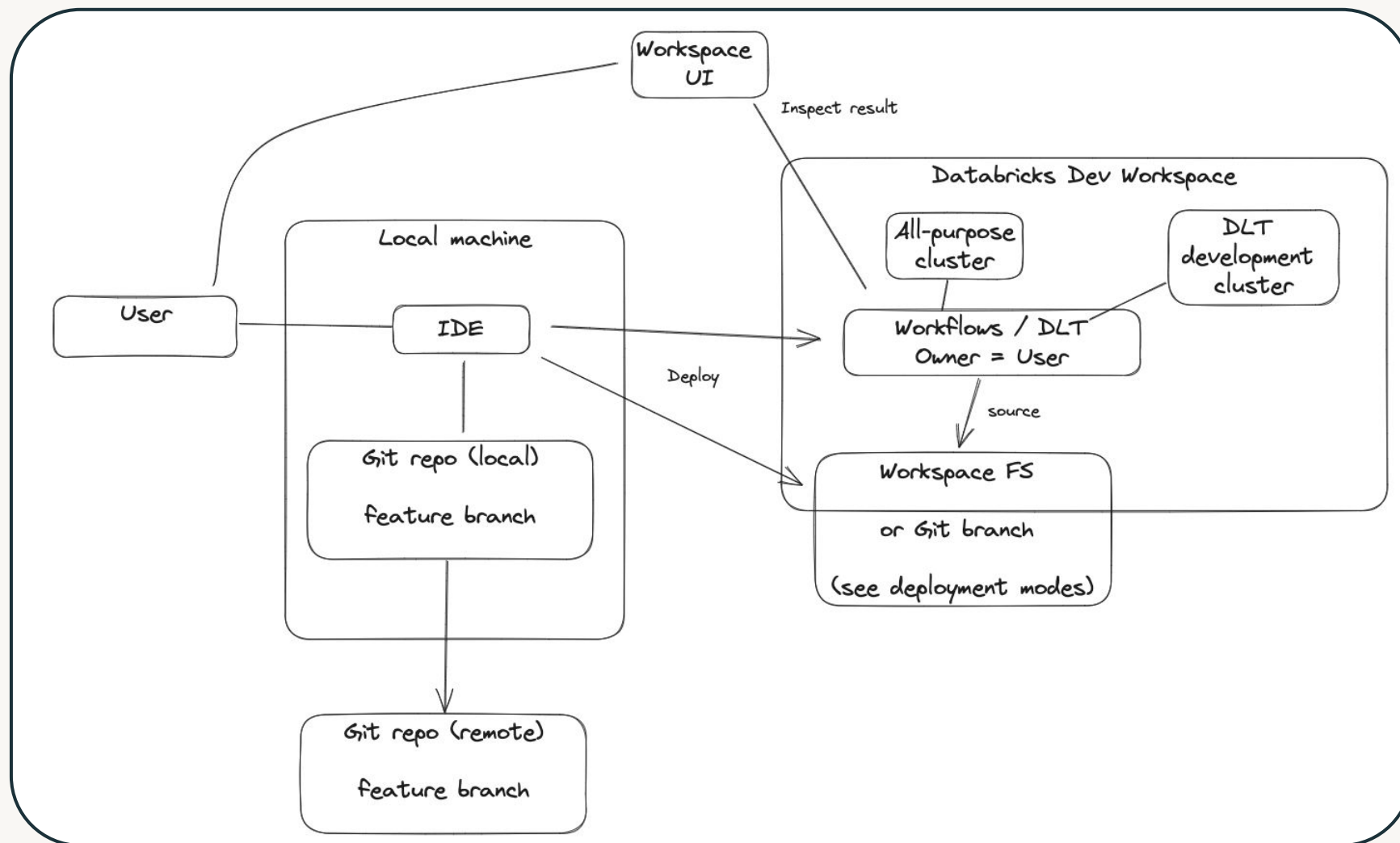


Code Development Workflow: IDE and UI Hybrid

- Pros:
 - Interactive development
 - Modularisation
 - Fast responds & iterations
 - Two way sync
- Cons:
 - Requires Repos
 - Context switching between IDE and UI
 - Requires notebook usage



Development Pattern: Deploy and Run

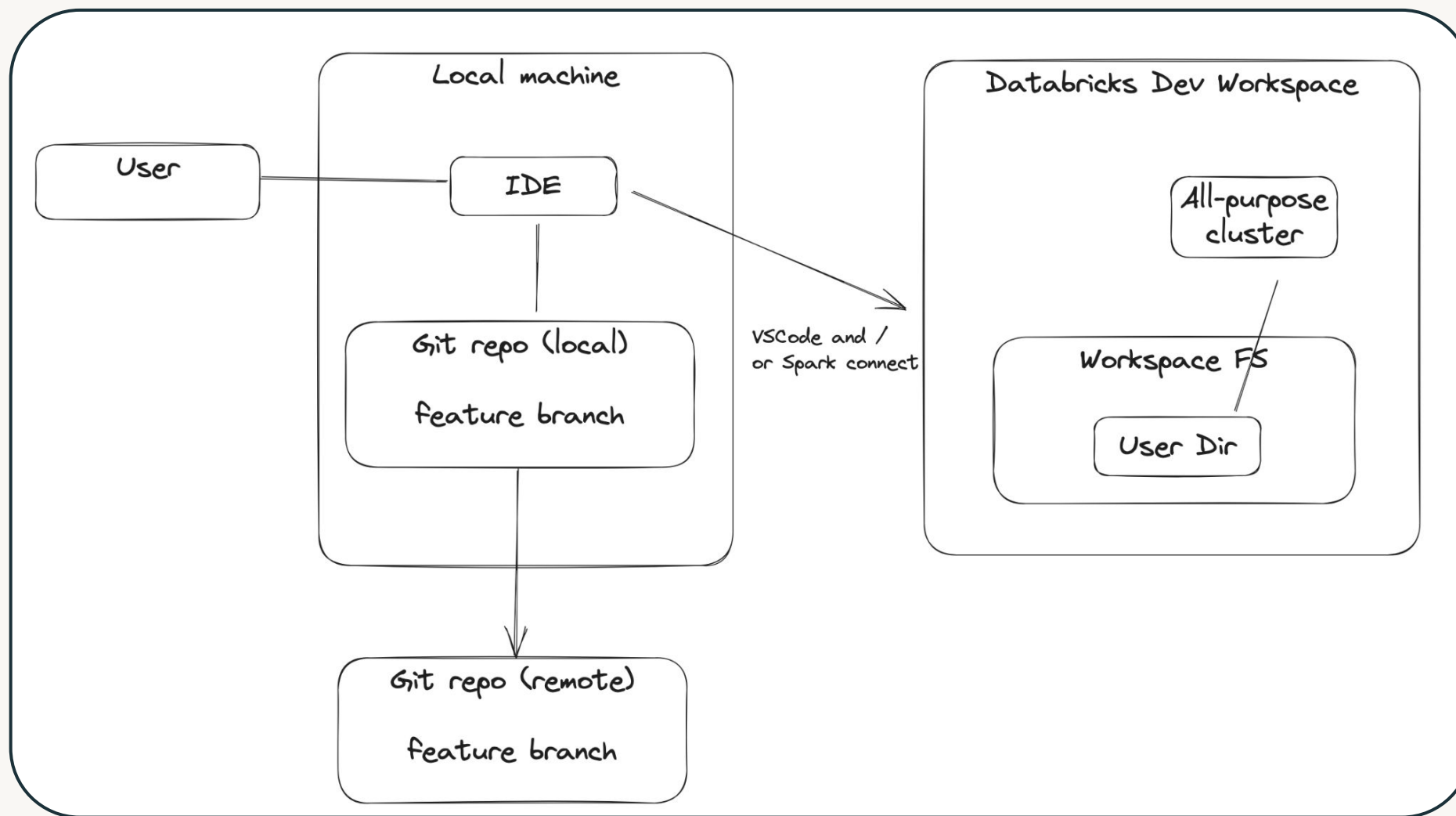


Code Development Workflow: Deploy and Run

- Pros:
 - Does not require Repos
 - Low context switching between IDE and UI
 - Applies to Delta Live Table development as well as regular Spark
 - Databricks job definition developed simultaneously
- Cons:
 - No interactive development – interactive manual testing at best
 - Longer feedback loops
 - Forced to create job definitions simultaneously



Development Pattern: IDE integration



Code Development Workflow: IDE integration

- Pros:
 - IDE support
 - Breakpoint debugging (debugging easier with highly modularized code) over Databricks Connect
 - Does not require Repos
 - No context switching
- Cons:
 - Environment shift between spark and non-spark with Databricks Connect
 - VS Code extension executes in batches – interactive only via Databricks Connect
 - Longer feedback loop
 - Not leveraging any Notebook or UI Features



Developing Job Definitions

- Applies to Databricks Workflows and Delta Live Tables
- Essentially like “Deploy and Run”
- Process
 - Develop workflow definition locally. Use Databricks UI “View Json” feature and documentation for help.
 - Run validation if supported (e.g. `terraform plan`, `databricks bundle validate`) from local terminal
 - Deploy from local terminal to dev
 - Inspect result definition in UI. Trigger run.



Developing and Executing Automated Tests

Unit testing

- Test code in your modules
- Code mostly contains functions that define data frame transformations, no/low Databricks specific features (dbutils, autoloader, ...).
 - Mock Databricks specific features
 - Use DBConnect to run Spark portion / DBX specific features on Databricks
- Run unit tests locally and on build agent during merge request
 - Compare result dataframe to expected using library (next slide)
- Install test-requirements.txt: Contains testing library, format checkers and libraries pre-installed on Databricks Runtime (spark, delta, ...)
- Use a readable format for your unit test data (small csv files, in-code)
 - Checked into Repository



Spark-specific challenges

- Specialized assertions:
 - Compare Dataframe content:
 - As whole, or individual columns
 - Ordered / not ordered
 - Approximate (floating point numbers)
 - Ignoring nulls or not
 - Compare Dataframe schema
- Popular strategies:
 - Native assertion functions in Spark 3.5 / DBR 13.3+
 - 3rd party testing libraries
 - toPandas()
 - Subtracting data frame from each other – [see](#)
- Specifically for Databricks – support for testing of code in notebooks
 - [Unit testing for notebooks](#)



Native Testing for PySpark in Spark 3.5 / DBR 13.3+

pyspark.testing.assertDataFrameEqual

pyspark.testing.assertDataFrameEqual(*actual*: Union[pyspark.sql.dataframe.DataFrame, pandas.DataFrame, [pyspark.pandas.DataFrame](#), List[pyspark.sql.types.Row]], *expected*: Union[pyspark.sql.dataframe.DataFrame, pandas.DataFrame, [pyspark.pandas.DataFrame](#), List[pyspark.sql.types.Row]], *checkRowOrder*: bool = False, *rtol*: float = 1e-05, *atol*: float = 1e-08)

A util function to assert equality between *actual* and *expected* (DataFrames or lists of Rows), with [\[source\]](#) optional parameters *checkRowOrder*, *rtol*, and *atol*.

Supports Spark, Spark Connect, pandas, and pandas-on-Spark DataFrames. For more information about pandas-on-Spark DataFrame equality, see the docs for [assertPandasOnSparkEqual](#).

 **New in version 3.5.0.**

pyspark.testing.assertPandasOnSparkEqual

pyspark.testing.assertPandasOnSparkEqual(*actual*: Union[pyspark.pandas.frame.DataFrame, pyspark.pandas.series.Series, pyspark.pandas.indexes.base.Index], *expected*: Union[pyspark.pandas.frame.DataFrame, pandas.core.frame.DataFrame, pyspark.pandas.series.Series, pandas.core.series.Series, pyspark.pandas.indexes.base.Index, pandas.core.indexes.base.Index], *checkExact*: bool = True, *almost*: bool = False, *rtol*: float = 1e-05, *atol*: float = 1e-08, *checkRowOrder*: bool = True) [\[source\]](#)

A util function to assert equality between *actual* (pandas-on-Spark object) and *expected* (pandas-on-Spark or pandas object).

 **New in version 3.5.0.**

pyspark.testing.assertSchemaEqual

pyspark.testing.assertSchemaEqual(*actual*: [pyspark.sql.types.StructType](#), *expected*: [pyspark.sql.types.StructType](#)) [\[source\]](#)

A util function to assert equality between DataFrame schemas *actual* and *expected*.

 **New in version 3.5.0.**

See [Example in Spark Docs](#)



Testing libraries for Spark

- [chispa](#) – Python version of spark-fast-tests
 - Authored by our own Matthew Powers
- [spark-testing-base](#):
 - Scala & Python support
 - Supports RDD, Dataframe/Dataset, Streaming APIs
- [spark-fast-tests](#) – Scala, Spark 2 & 3
- [pytest-spark](#) – Python, native integration with pytest
- **Code samples for all libraries in [one place](#)**

```
from chispa.dataframe_comparer import assert_df_equality

def test_remove_non_word_characters_long():
    source_data = [
        ("jo&&se",),
        ("**li**",),
        ("#:luisa",),
        (None,)
    ]
    source_df = spark.createDataFrame(source_data, ["name"])

    actual_df = source_df.withColumn(
        "clean_name",
        remove_non_word_characters(F.col("name"))
    )

    expected_data = [
        ("jo&&se", "jose"),
        ("**li**", "li"),
        ("#:luisa", "luisa"),
        (None, None)
    ]
    expected_df = spark.createDataFrame(expected_data, ["name", "clean_name"])

    assert_df_equality(actual_df, expected_df)
```

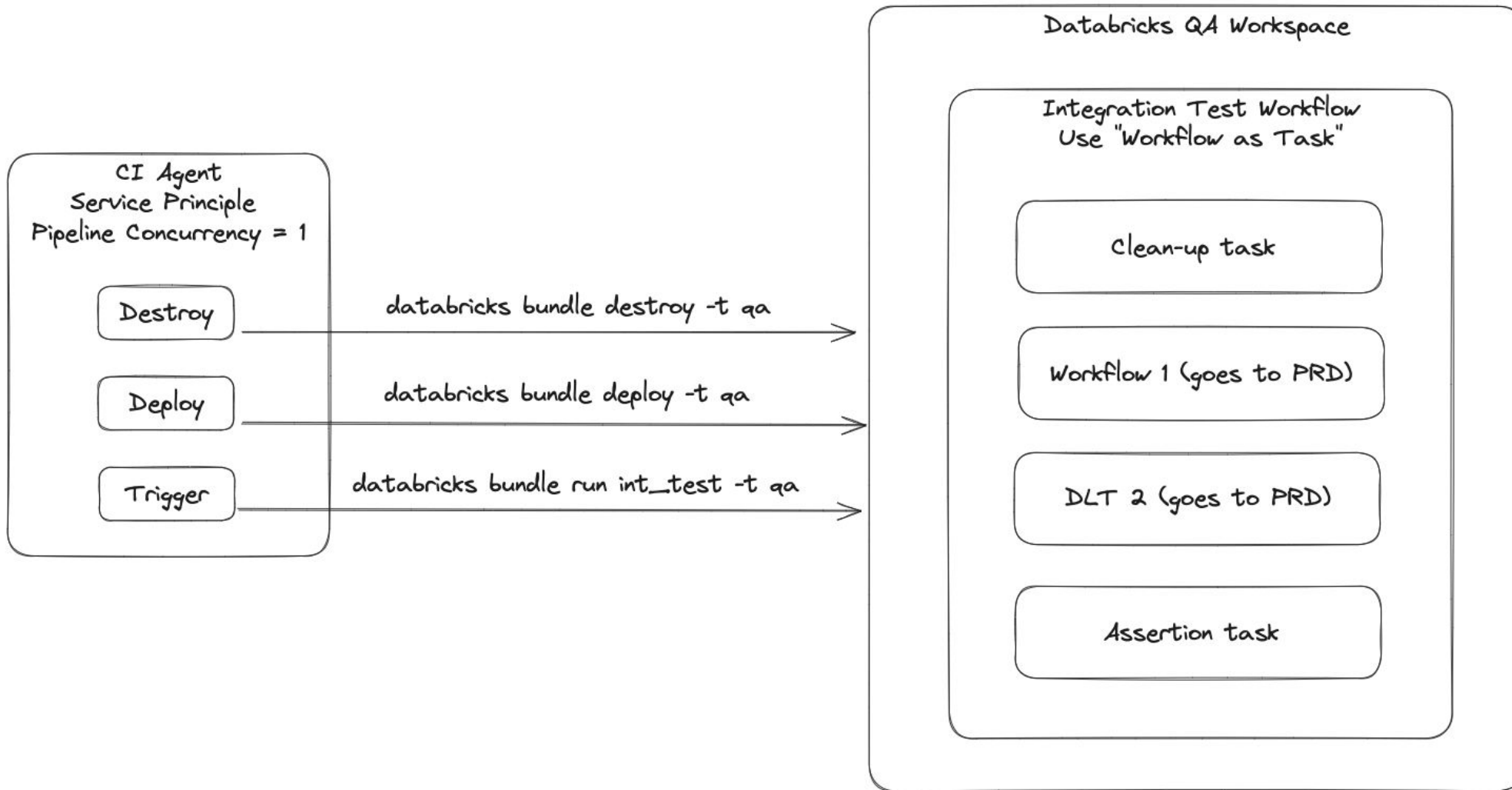


Integration test

- Integration test = execution of all workflows of the data product incl. result assertions
- Run on CI agent during MR using service principal usually on separate workspace
- Use subset of data and configurations to speed up execution
- Set pipeline concurrency to 1 to avoid conflicts
 - Separating deployment in one workspace for the same user (SP) is a lot of additional work and maintenance
- CI deploys and triggers a integration test workflow that uses “Workflow as Task” to execute actual workflows. Integration test should cleanup possible previous results upfront and assert at the end.
- Execute all workflows in right order, parallelize workflows if possible
- Trade Off: Time and simplicity vs comprehensiveness



Integration test



Additional information

Documentation

- Official Databricks Terraform Provider Page on [Terraform Registry](#)
- Terraform guide on [Databricks documentation site](#)
- [Create clusters, notebooks and jobs with Terraform](#)
- [Databricks CLI](#)
- [Databricks Asset Bundles](#)



Examples and blog posts

- [Databricks Terraform Examples Repository](#) demonstrates use of Terraform in CI/CD pipelines
- [Best practices for using Databricks with an IDE](#)
- [Databricks Workflows Through Terraform](#)
- [CI/CD workflows with Git integration and Databricks Repos](#)
- [Automate continuous integration and continuous delivery on Databricks using Databricks Labs CI/CD Templates](#)
- [Implementing CI/CD on Databricks Using Databricks Notebooks and Azure DevOps](#)
- [Continuous integration and delivery on Azure Databricks using Azure DevOps](#)



Recommended books on DevOps & CI/CD

- [Clean Code: A Handbook of Agile Software Craftsmanship](#) by Robert C. Martin
- [Refactoring: Improving the Design of Existing Code](#) by Martin Fowler, Kent Beck, Don Roberts
- [Test-Driven Development: By Example](#) by Kent Beck
- [Code Complete, 2nd ed.](#) by Steve McConnell
- [The Pragmatic Programmer: From Journeyman to Master](#) by Andy Hunt, Dave Thomas





databricks